

# **A LONG-TERM INTERNSHIP REPORT**

**On**

## **Machine Learning Using Python**

**Submitted to Department of Computer Science**

**By**

**SRAVANI DEVAGUPTAPU**

**III BCA**

**Under the Esteemed Guidance of**

**MRS.BHAVANA SRAVANTHI**



**LECTURER**

**Department of Computer Science**

**ADITYA DEGREE COLLEGE FOR WOMEN,**

**GOPALAPATNAM**



**ADHOC NETWORK**

Tech Company

# CERTIFICATE Of Internship

Proudly Presented To:

**SRAVANI DEVAGUPTAPU**

a student of Aditya Degree College, has successfully  
completed Machine Learning Using Python  
Internship at Adhoc Network Tech Company from  
February 08, 2024 to April 20, 2024.



Project Coordinator

# ADITYA DEGREE COLLEGE

## Department of Computer Science



### CERTIFICATE

This is to certify that The Long Term Internship entitled,

“Machine Learning Using Python” is a bonified work of **Sravani Devaguptapu**, bearing **121127306037**, III BCA, submitted to the Department of Computer Science, Aditya Degree College for women, Gopalapatnam for the academic year 2021-2024.

**Internship Guide**

**Miss.N. SANDHYA , B.Tech**

**Project Coordinator**

**Adhoc Network Tech Company**

**Head of the Department**

**Ms.K Bhavana Sravanthi**

**External Examiner**

**Principal**

# ADITYA DEGREE COLLEGE

## Department of Computer Science



### DECLARATION BY THE STUDENT

I hereby declare that the work described in this Long Term Internship, entitled “**Machine Learning Using Python**” which is being submitted by me in partial fulfillment of the requirements for the award of degree of **Bachelor of Computer Applications** (BCA) from Aditya Degree College for women, Gopalapatnam under the guidance of **N.SANDHYA** , Project Coordinator of Adhoc Network Tech Company, India and Canada.

Place: Gopalapatnam

Date: 20-04-2024

# ADITYA DEGREE COLLEGE

## Department of Computer Science



### CERTIFICATE FROM THE SUPERVISOR

This is to certify that the Long Term Internship entitled, "**Machine Learning Using Python**", that is being submitted by **Sravani Devaguptapu** bearing **121127306037, III BCA**, which is being submitted by me in partial fulfillment of the requirements for the award of degree of Bachelor of Computer Applications from **Aditya Degree College for women**, bonafied work carried out by him under my guidance and Supervision.

MRS.M.RADHA

# ACKNOWLEDGEMENT

No endeavour is completed without the valuable support of others. I would like to take this opportunity to extend my sincere gratitude to all those who have contributed to the successful completion of this Short-Term Internship Project Report.

At this juncture I feel deeply honoured in expressing my sincere thanks to Miss **.DEVIKA PAKRUTHI**, Founder And CEO of Adhoc Network Tech Company, India and Canada for making the resources available at right time and providing valuable insights leading to the successful completion of my Short-Term Internship Project Report.

It is privilege to thank **Dr.N.SESHA REDDY**, Chairman, Aditya group of institutions for providing state-of-the-Art facilities, experienced and talented faculty members.

It is privilege to thank **Dr. N. SUGUNA REDDY**, Secretary Madam, Aditya group of institutions for providing Short-Term Internship Project Report from Adhoc.

I thank **Dr.B.E.V.L.NAIDU**, Academic Director, Aditya Degree College for his continuous support and encouragement in my endeavour.

I express my deep sense of gratitude to **Mr. N.H.Satish Kumar, Principal**, for his Efforts and for giving us permission for carrying out this Long-Term Internship.

I thank **MRS.K.Bhavna Sravanthi** Head of the Department of Bachelor of Computer Science, Aditya Degree College-Visakhapatnam(Gopalapatnam), for supporting and encouraging me in completion of my Long-Term Internship.

Finally I thank all the faculty members of our Department who contributed their valuable suggestions in completion of Long-Term Internship report and I also put my sincere thanks to My Parents who stood with me during the whole Long-Term Internship.

- *Sravani Devaguptapu*

# ABOUT ADITYA DEGREE COLLEGES



**Dr.N.SESHA REDDY**  
CHAIRMAN



**Dr.SUGUNA REDDY**  
SECRETARY



**Dr.B.E.V.L. NAIDU**  
ACADEMIC DIRECTOR

Aditya Degree colleges are the precious gifts presented to the twin Godavari Districts by ADITYA. Educational group ADITYA Degree College which was established in 1998 in Kakinada fulfilled the hopes and aspirations of many graduates and had been acclaimed as the best degree college under Andhra University. Encouraged by the 100% result in 2003, ADITYA added several feathers to its cap by launching Degree Colleges in Rajahmundry in 2003, in Vizag and Palakol in 2005 and in Tatipaka in 2006.

Needless to say, in the present scenario girls excel more than boys in education and they give tough competition to boys. Owing to their diffidence and inhibition, girls find difficulty in expressing their doubts in a classroom of Co-ed College. Moreover, parents have many objections in sending their daughters to a co-ed college. Realizing this, ADITYA successfully leads Degree colleges for girls to prove their talents in curricular, co-curricular and extra-curricular activities. ADITYA started P.G College also for Women to encourage them for higher education.

## VISION

To provide inclusive education with innovative methods and strenuous efforts for inculcating human values, professionalism and scientific instillation in the realm of Degree Education to all sections of students irrespective of race, region and religion with special focus to stand independently and to emerge as centre for Research and Development.

## MISSION

To provide ample scope for multifaceted development of local youth. To provide quality higher education to student community. To Recruit Highly Qualified and Experienced Faculty to provide Quality Education.

## ABOUT ADHOC NETWORK TECH COMPANY - INDIA,CANADA



**CEO Devika Pakruthi** is a highly accomplished and recognized entrepreneur who has garnered prestigious awards at various levels for her outstanding contributions to the business world.

### **Awards that Devika Pakruthi has won:**

- Young Entrepreneur Award - City Level (Visakhapatnam)
- Best Ongoing Startup - State Level (Andhra Pradesh)
- Women Rising Star of the Year- National Level (New Delhi)
- Youngest CEO of the Year - State Representation (Karnataka)

Adhoc Network is started in the year 2020 at Visakhapatnam by an young Women entrepreneur Miss. Devika Pakruthi Founder & CEO with an intention to provide employment opportunities to the youth and also to impart the best quality training and practical exposure to the Students which enhances their employability Skills. Her journey started and collapsed with the wide spread of Covid-19 but her determination and aspirations made her journey more futuristic and she never gave up the thought to GIVE-UP. This is where Devika made her dreams come true and alive. She never expected with a sole objective of making Profit but her determination to impart the quality training made her to reach the peaks of success at the young age.

Adhoc Network-we are proud to have been awarded a Hattrick of Awards. This Achievement is a testament to our commitment to Excellence and Innovation in the Software development Best heading company in the market.

Devika Pakruthi, a name synonymous with innovation, empowerment, and success. As the proud recipient of the Young Entrepreneur Award, Best Women- Led Startup Award, Women Rising Star of the Year Award and Youngest CEO of the Year Award, Devika Pakruthi has etched her name into the annals of contemporary business history.

## Vision, Mission and values of the Organization:-

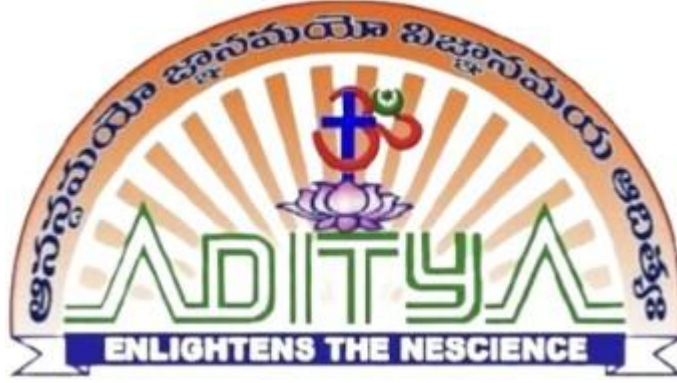
### VISION:-

Our vision is to be a leading global provider of interactive and reliable Software Solution empowering business to Thrive in the digital age.

### MISSION:-

Our mission is to develop cutting-edge Software Solutions that Solve complex business challenges, enhance Operations efficiency and drive Sustainable growth for our client we strive to deliver exceptional value by leveraging emerging technologies, fostering Strategic partnerships and maintaining a Customer, Centric approach Values

- Innovation
- Excellence
- Collaboration
- Integrity
- Customer Centricity
- Continuous Learning



**BATCH NO:-5**

**TEAM MEMBERS NAMES:**

|                                 |              |
|---------------------------------|--------------|
| 1.SRAVANI DEVAGUPTAPU           | 121127306037 |
| 2.GOWTHAMI ALLURI               | 121127306006 |
| 3.PRAGNA KANDREGULA             | 121127306071 |
| 4.JOSHNA SARVASUDDI             | 121127306144 |
| 5.ANKITHA SRI DUKKA             | 121127306008 |
| 6.MAJJI YASASWINI               | 121127306093 |
| 7.BHAVNEETH KAUR DHILLON        | 121127306038 |
| 8.GOWRI NAGALAKSHMI INDUGUBILLI | 121127306061 |

# INDEX

| <b>S.NO</b> | <b>TITLE</b>                       | <b>PAGE<br/>NUMBER</b> |
|-------------|------------------------------------|------------------------|
| <b>1</b>    | <b>Abstract</b>                    | <b>1-2</b>             |
| <b>2</b>    | <b>Introduction</b>                | <b>3-59</b>            |
| <b>3</b>    | <b>Literature survey</b>           | <b>60-64</b>           |
| <b>4</b>    | <b>System study</b>                | <b>65-66</b>           |
| <b>5</b>    | <b>System requirements</b>         | <b>67</b>              |
| <b>6</b>    | <b>System Design</b>               | <b>68-76</b>           |
| <b>7</b>    | <b>Software Environment</b>        | <b>77-91</b>           |
| <b>8</b>    | <b>Implementation</b>              | <b>92-99</b>           |
| <b>9</b>    | <b>Sample Code</b>                 | <b>100-123</b>         |
| <b>10</b>   | <b>Input And Output<br/>Design</b> | <b>124-126</b>         |
| <b>11</b>   | <b>System Analysis</b>             | <b>127-134</b>         |
| <b>12</b>   | <b>System Testing</b>              | <b>135-140</b>         |
| <b>13</b>   | <b>Screenshots</b>                 | <b>141-147</b>         |
| <b>14</b>   | <b>Conclusion</b>                  | <b>148-151</b>         |
| <b>15</b>   | <b>Bibliography</b>                | <b>152-154</b>         |

# **ABSTRACT**

## **OUR PROPOSED PROJECT ABSTRACT:**

In the realm of healthcare, timely and accurate drug recommendations during medical emergencies can significantly impact patient outcomes. This project presents a robust "Drug Recommendation System in Medical Emergencies using Machine Learning," implemented in Python. The system leverages two powerful classification algorithms, namely the Random Forest Classifier and the Decision Tree Classifier, attaining remarkable accuracies of 100% on both training and test datasets.

The dataset employed in this project comprises 1200 records, each characterized by 30 features. These features encapsulate a diverse set of medical parameters, providing a comprehensive representation of patient health. The dataset spans 10 distinct classes, encompassing a spectrum of medical conditions: Allergy, Chickenpox, Chronic, Cold, Diabetes, Fungal, GERD, Jaundice, Malaria, and Pneumonia.

The Random Forest Classifier, known for its ensemble learning capabilities, and the Decision Tree Classifier, recognized for its interpretability, were meticulously chosen to model the intricate relationships within the dataset. Both algorithms exhibited exceptional performance, achieving perfect accuracy scores on both training and test datasets, signifying the efficacy of the developed recommendation system.

This project not only serves as a testament to the potency of machine learning in healthcare applications but also underscores the critical role of accurate drug recommendations in emergency medical scenarios. The achieved 100% accuracy

underscores the reliability and precision of the system, instilling confidence in its potential deployment in real-world medical settings. As we navigate the intersection of technology and healthcare, this Drug Recommendation System stands as a testament to the transformative impact of machine learning on patient care in critical situations.

## **ABSTRACT**

### **OUR PROPOSED ABSTRACT:**

Stress, an increasingly prevalent aspect of modern life, can significantly impact an individual's physical and mental well-being. Hence, understanding and monitoring stress levels play a crucial role in promoting overall health and quality of life. The project "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier" presents a novel and effective approach to detect human stress levels by analyzing their sleeping habits. Leveraging the powerful capabilities of Python programming language, the study employs the Random Forest Classifier algorithm, known for its versatility and accuracy in classification tasks. The primary objective of this research is to develop a reliable stress detection system that can provide valuable insights into individuals' stress levels, enabling timely interventions and promoting better mental health. The dataset used in this study is carefully curated and comprises various essential parameters related to both sleep patterns and stress levels. These parameters include the user's snoring range, respiration rate, body temperature, limb movement rate, blood oxygen levels, eye movement, the number of hours of sleep, heart rate, and stress levels categorized into five classes: 0 (low/normal), 1 (medium low), 2 (medium), 3 (medium high), and 4 (high). The inclusion of these diverse parameters ensures a comprehensive analysis of sleep patterns and their correlation with stress levels. To achieve accurate stress detection, the Random Forest Classifier is chosen as the machine learning model due to its ability to handle complex data relationships, mitigate overfitting, and offer high predictive accuracy. The model is trained using the dataset, and its performance is evaluated on a separate test dataset to ensure generalization and unbiased assessment. The results of the experiments reveal a Training score of 100% and an impressive Test

score of 97%, demonstrating the effectiveness and robustness of the proposed methodology. The achieved high accuracy showcases the model's capability to learn intricate patterns from the dataset and make accurate stress predictions based on the user's sleeping habits. This stress detection system's potential applications are vast, ranging from personal health monitoring to medical research and interventions. By enabling individuals to gain insights into their stress levels through analysis of their sleep habits, the system empowers them to take proactive measures to alleviate stress, improve sleep quality, and foster overall well-being.

# INTRODUCTION

## **Introduction to Human Stress:**

Stress is an integral part of human life, an intricate response to various external and internal demands that challenge an individual's physical, emotional, or psychological equilibrium. As humans navigate through the complexities of daily life, they encounter numerous stressors, ranging from work pressures, personal relationships, financial burdens, to health-related concerns, and more. While stress is a natural survival mechanism that prepares the body for fight or flight responses in threatening situations, prolonged or excessive stress can have detrimental effects on overall well-being.

The human body responds to stress through a series of physiological and hormonal reactions, often referred to as the stress response or "fight-or-flight" response. During stressful situations, the adrenal glands release hormones such as cortisol and adrenaline, triggering increased heart rate, elevated blood pressure, heightened senses, and redirection of energy to critical functions.

While short-term stress can be adaptive and mobilize resources to tackle challenges effectively, chronic stress can lead to a range of health issues. Mental health conditions like anxiety and depression, cardiovascular problems, immune system suppression, and impaired cognitive function are among the consequences of prolonged stress.

Understanding and managing stress play a crucial role in maintaining physical and mental well-being. Identifying individual stress patterns, triggers, and stress

management techniques are essential for promoting resilience and coping effectively with life's challenges.

Advancements in technology and data analysis have opened up new possibilities for stress detection and management. With the advent of wearable devices and health monitoring systems, it is now possible to collect extensive data on individuals' physiological and behavioral responses to stress. Machine learning algorithms, such as Random Forest Classifier and Naive Bayes, offer powerful tools for analyzing these data and predicting stress levels based on various parameters.

This project, "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier," aims to explore the relationship between sleep patterns and stress levels. By leveraging machine learning techniques and sleep-related data, the project seeks to develop an accurate and efficient system for stress detection. The insights gained from this project can provide individuals with valuable information about their stress levels, facilitating timely interventions and proactive stress management.

In conclusion, human stress is a complex phenomenon that impacts individuals' lives in diverse ways. The ability to identify and manage stress is crucial for maintaining physical and mental health. The project's objective of stress detection based on sleeping habits using machine learning holds the potential to contribute to stress management strategies and enhance overall well-being. By leveraging technology and data analysis, the project endeavors to provide individuals with the tools they need to better understand and cope with stress, fostering a healthier and more resilient society.

## **What is Machine Learning?**

Machine Learning is a system of computer algorithms that can learn from example through self-improvement without being explicitly coded by a programmer. Machine learning is a part of artificial Intelligence which combines data with statistical tools to predict an output which can be used to make actionable insights.

The breakthrough comes with the idea that a machine can singularly learn from the data (i.e., example) to produce accurate results. Machine learning is closely related to data mining and Bayesian predictive modeling. The machine receives data as input and uses an algorithm to formulate answers.

A typical machine learning tasks are to provide a recommendation. For those who have a Netflix account, all recommendations of movies or series are based on the user's historical data. Tech companies are using unsupervised learning to improve the user experience with personalizing recommendation.

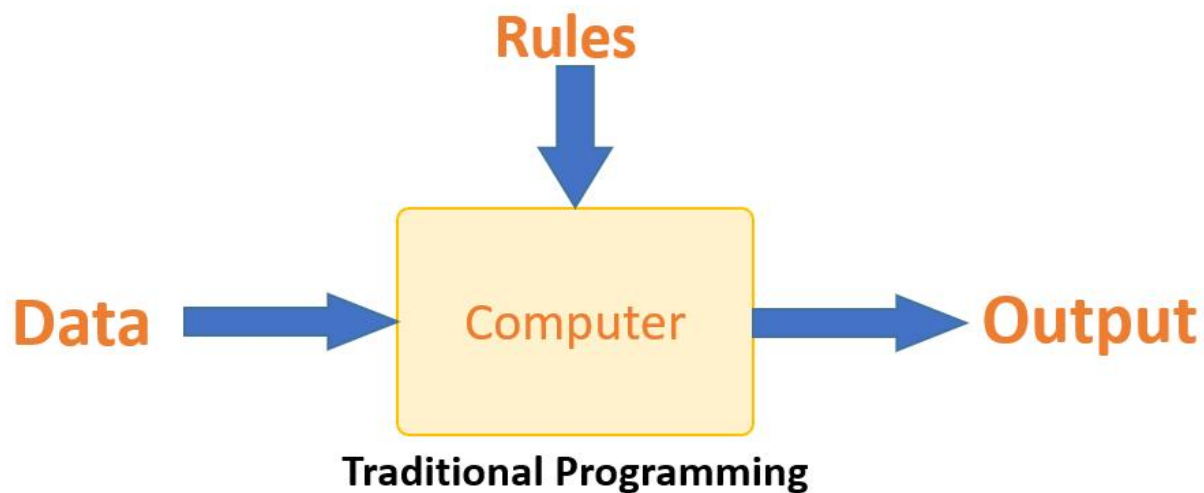
Machine learning is also used for a variety of tasks like fraud detection, predictive maintenance, portfolio optimization, automatize task and so on.

## **Machine Learning vs. Traditional Programming**

Traditional programming differs significantly from machine learning. In traditional programming, a programmer code all the rules in consultation with an expert in the industry for which software is being developed. Each rule is based on a logical foundation; the machine will execute an output following the logical statement.

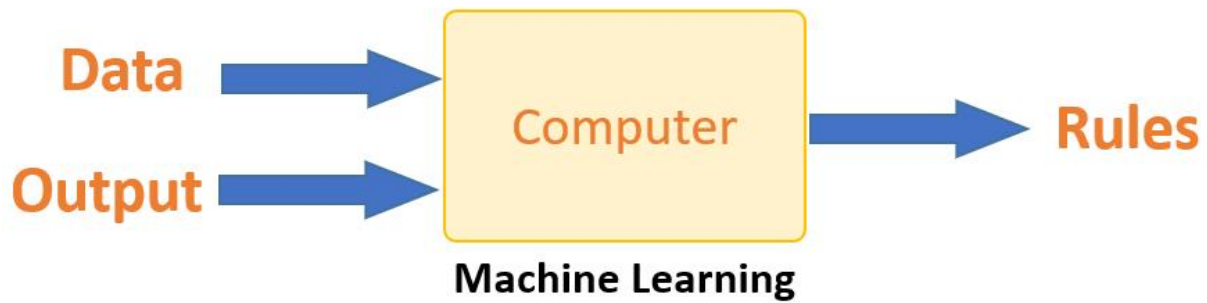
When the system grows complex, more rules need to be written. It can quickly become unsustainable to maintain.

Traditional programming differs significantly from machine learning. In traditional programming, a programmer code all the rules in consultation with an expert in the industry for which software is being developed. Each rule is based on a logical foundation; the machine will execute an output following the logical statement. When the system grows complex, more rules need to be written. It can quickly become unsustainable to maintain.



Traditional Programming

Machine learning is supposed to overcome this issue. The machine learns how the input and output data are correlated and it writes a rule. The programmers do not need to write new rules each time there is new data. The algorithms adapt in response to new data and experiences to improve efficacy over time.



Machine Learning

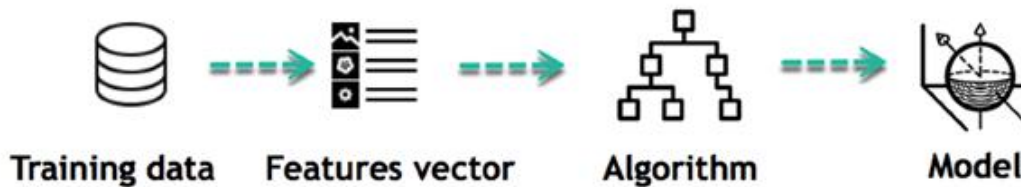
### How does Machine Learning Work?

Machine learning is the brain where all the learning takes place. The way the machine learns is similar to the human being. Humans learn from experience. The more we know, the more easily we can predict. By analogy, when we face an unknown situation, the likelihood of success is lower than the known situation. Machines are trained the same. To make an accurate prediction, the machine sees an example. When we give the machine a similar example, it can figure out the outcome. However, like a human, if its feed a previously unseen example, the machine has difficulties to predict.

The core objective of machine learning is the **learning** and **inference**. First of all, the machine learns through the discovery of patterns. This discovery is made thanks to the **data**. One crucial part of the data scientist is to choose carefully which data to provide to the machine. The list of attributes used to solve a problem is called a **feature vector**. You can think of a feature vector as a subset of data that is used to tackle a problem.

The machine uses some fancy algorithms to simplify the reality and transform this discovery into a **model**. Therefore, the learning stage is used to describe the data and summarize it into a model.

### Learning Phase



For instance, the machine is trying to understand the relationship between the wage of an individual and the likelihood to go to a fancy restaurant. It turns out the machine finds a positive relationship between wage and going to a high-end restaurant: This is the model

### Inferring

When the model is built, it is possible to test how powerful it is on never-seen-before data. The new data are transformed into a features vector, go through the model and give a prediction. This is all the beautiful part of machine learning. There is no need to update the rules or train again the model. You can use the model previously trained to make inference on new data.

### Inference from Model



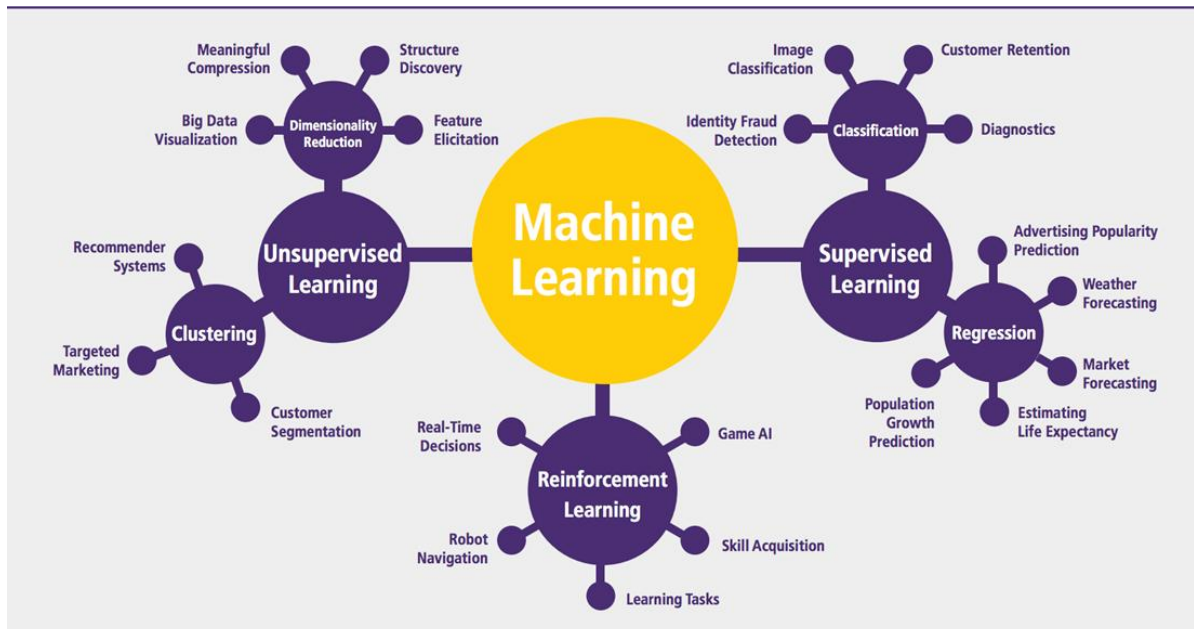
The life of Machine Learning programs is straightforward and can be summarized in the following points:

1. Define a question
2. Collect data
3. Visualize data

4. Train algorithm
5. Test the Algorithm
6. Collect feedback
7. Refine the algorithm
8. Loop 4-7 until the results are satisfying
9. Use the model to make a prediction

Once the algorithm gets good at drawing the right conclusions, it applies that knowledge to new sets of data.

## Machine Learning Algorithms and Where they are Used?



### Machine learning Algorithms

Machine learning can be grouped into two broad learning tasks: Supervised and Unsupervised. There are many other algorithms

### Supervised learning

An algorithm uses training data and feedback from humans to learn the relationship of given inputs to a given output. For instance, a practitioner can use marketing expense and weather forecast as input data to predict the sales of cans.

You can use supervised learning when the output data is known. The algorithm will predict new data.

There are two categories of supervised learning:

- Classification task
- Regression task

## **Classification**

Imagine you want to predict the gender of a customer for a commercial. You will start gathering data on the height, weight, job, salary, purchasing basket, etc. from your customer database. You know the gender of each of your customer, it can only be male or female. The objective of the classifier will be to assign a probability of being a male or a female (i.e., the label) based on the information (i.e., features you have collected). When the model learned how to recognize male or female, you can use new data to make a prediction. For instance, you just got new information from an unknown customer, and you want to know if it is a male or female. If the classifier predicts male = 70%, it means the algorithm is sure at 70% that this customer is a male, and 30% it is a female.

The label can be of two or more classes. The above Machine learning example has only two classes, but if a classifier needs to predict object, it has dozens of classes (e.g., glass, table, shoes, etc. each object represents a class)

## **Regression**

When the output is a continuous value, the task is a regression. For instance, a financial analyst may need to forecast the value of a stock based on a range of features like equity, previous stock performances, macroeconomics index. The system will be trained to estimate the price of the stocks with the lowest possible error.

| Algorithm Name             | Description                                                                                                                                                                                                                             | Type                      |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <b>Linear regression</b>   | Finds a way to correlate each feature to the output to help predict future values.                                                                                                                                                      | Regression                |
| <b>Logistic regression</b> | Extension of linear regression that's used for classification tasks. The output variable is binary (e.g., only black or white) rather than continuous (e.g., an infinite list of potential colors)                                      | Classification            |
| <b>Decision tree</b>       | Highly interpretable classification or regression model that splits data-feature values into branches at decision nodes (e.g., if a feature is a color, each possible color becomes a new branch) until a final decision output is made | Classification/Regression |
| <b>Naive Bayes</b>         | The Bayesian method is a classification method that makes use of the Bayesian theorem. The theorem updates the prior knowledge of an event with the independent probability of each feature that can affect the event.                  | Classification            |

| Algorithm Name                 | Description                                                                                                                                                                                                                                                                                                                                                                                            | Type                         |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <b>Support vector machine</b>  | Support Vector Machine, or SVM, is typically used for the classification task. SVM algorithm finds a hyperplane that optimally divides the classes. It is best used with a non-linear solver.                                                                                                                                                                                                          | Regression<br>Classification |
| <b>Random forest</b>           | The algorithm is built upon a decision tree to improve the accuracy drastically. Random forest generates many times simple decision trees and uses the 'majority vote' method to decide on which label to return. For the classification task the final prediction will be the one with the most vote; while for the regression task, the average prediction of all the trees is the final prediction. | Regression<br>Classification |
| <b>AdaBoost</b>                | Classification or regression technique that uses a multitude of models to come up with a decision but weighs them based on their accuracy in predicting the outcome                                                                                                                                                                                                                                    | Regression<br>Classification |
| <b>Gradient-boosting trees</b> | Gradient-boosting trees is a state-of-the-art classification/regression technique. It is focusing on the error committed by the previous trees and tries to correct it.                                                                                                                                                                                                                                | Regression<br>Classification |

## Unsupervised learning

In unsupervised learning, an algorithm explores input data without being given an explicit output variable (e.g., explores customer demographic data to identify patterns)

You can use it when you do not know how to classify the data, and you want the algorithm to find patterns and classify the data for you

| Algorithm                      | Description                                                                                                                                        | Type                |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| <b>K-means clustering</b>      | Puts data into some groups (k) that each contains data with similar characteristics (as determined by the model, not in advance by humans)         | Clustering          |
| <b>Gaussian mixture model</b>  | A generalization of k-means clustering that provides more flexibility in the size and shape of groups (clusters)                                   | Clustering          |
| <b>Hierarchical clustering</b> | Splits clusters along a hierarchical tree to form a classification system. Can be used for Cluster loyalty-card customer                           | Clustering          |
| <b>Recommender system</b>      | Help to define the relevant data for making a recommendation.                                                                                      | Clustering          |
| <b>PCA/T-SNE</b>               | Mostly used to decrease the dimensionality of the data. The algorithms reduce the number of features to 3 or 4 vectors with the highest variances. | Dimension Reduction |

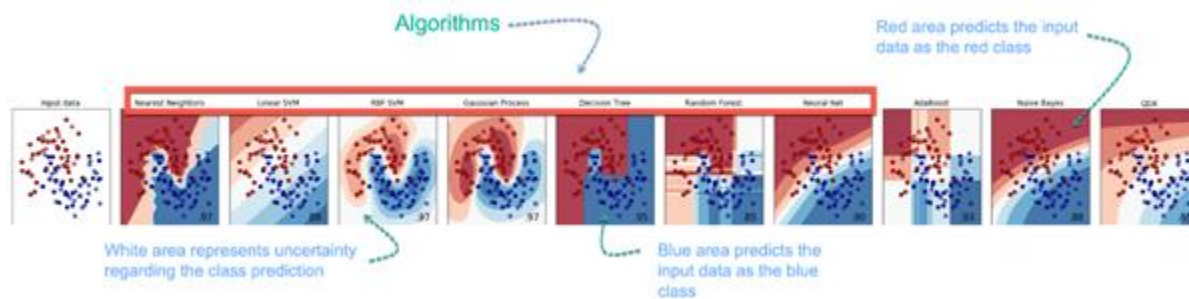
## How to Choose Machine Learning Algorithm

### Machine Learning (ML) algorithm:

There are plenty of machine learning algorithms. The choice of the algorithm is based on the objective.

In the Machine learning example below, the task is to predict the type of flower among the three varieties. The predictions are based on the length and the width of the petal. The picture depicts the results of ten different algorithms. The picture on the top left is the dataset. The data is classified into three categories: red, light blue and dark blue. There are some groupings. For instance, from the second image, everything in the upper left belongs to the red category, in the middle part, there is a mixture of uncertainty and light blue while the bottom corresponds to the dark

category. The other images show different algorithms and how they try to classified the data.



## Challenges and Limitations of Machine Learning

The primary challenge of machine learning is the lack of data or the diversity in the dataset. A machine cannot learn if there is no data available. Besides, a dataset with a lack of diversity gives the machine a hard time. A machine needs to have heterogeneity to learn meaningful insight. It is rare that an algorithm can extract information when there are no or few variations. It is recommended to have at least 20 observations per group to help the machine learn. This constraint leads to poor evaluation and prediction.

## Application of Machine Learning

### Augmentation:

- Machine learning, which assists humans with their day-to-day tasks, personally or commercially without having complete control of the output. Such machine learning is used in different ways such as Virtual Assistant, Data analysis, software solutions. The primary user is to reduce errors due to human bias.

### Automation:

- Machine learning, which works entirely autonomously in any field without the need for any human intervention. For example, robots performing the essential process steps in manufacturing plants.

### **Finance Industry**

- Machine learning is growing in popularity in the finance industry. Banks are mainly using ML to find patterns inside the data but also to prevent fraud.

### **Government organization**

- The government makes use of ML to manage public safety and utilities. Take the example of China with the massive face recognition. The government uses Artificial intelligence to prevent jaywalker.

### **Healthcare industry**

- Healthcare was one of the first industry to use machine learning with image detection.

### **Marketing**

- Broad use of AI is done in marketing thanks to abundant access to data. Before the age of mass data, researchers develop advanced mathematical tools like Bayesian analysis to estimate the value of a customer. With the boom of data, marketing department relies on AI to optimize the customer relationship and marketing campaign.

### **Example of application of Machine Learning in Supply Chain**

Machine learning gives terrific results for visual pattern recognition, opening up many potential applications in physical inspection and maintenance across the entire supply chain network.

Unsupervised learning can quickly search for comparable patterns in the diverse dataset. In turn, the machine can perform quality inspection throughout the logistics hub, shipment with damage and wear.

For instance, IBM's Watson platform can determine shipping container damage. Watson combines visual and systems-based data to track, report and make recommendations in real-time.

In past year stock manager relies extensively on the primary method to evaluate and forecast the inventory. When combining big data and machine learning, better forecasting techniques have been implemented (an improvement of 20 to 30 % over traditional forecasting tools). In term of sales, it means an increase of 2 to 3 % due to the potential reduction in inventory costs.

### **Example of Machine Learning Google Car**

For example, everybody knows the Google car. The car is full of lasers on the roof which are telling it where it is regarding the surrounding area. It has radar in the front, which is informing the car of the speed and motion of all the cars around it. It uses all of that data to figure out not only how to drive the car but also to figure out and predict what potential drivers around the car are going to do. What's impressive is that the car is processing almost a gigabyte a second of data.

### **Why is Machine Learning Important?**

Machine learning is the best tool so far to analyze, understand and identify a pattern in the data. One of the main ideas behind machine learning is that the computer can be trained to automate tasks that would be exhaustive or impossible for a human being. The clear breach from the traditional analysis is that machine learning can take decisions with minimal human intervention.

Take the following example for this ML tutorial; a retail agent can estimate the price of a house based on his own experience and his knowledge of the market.

A machine can be trained to translate the knowledge of an expert into features. The features are all the characteristics of a house, neighborhood, economic environment, etc. that make the price difference. For the expert, it took him

probably some years to master the art of estimate the price of a house. His expertise is getting better and better after each sale.

For the machine, it takes millions of data, (i.e., example) to master this art. At the very beginning of its learning, the machine makes a mistake, somehow like the junior salesman. Once the machine sees all the example, it got enough knowledge to make its estimation. At the same time, with incredible accuracy. The machine is also able to adjust its mistake accordingly.

Most of the big company have understood the value of machine learning and holding data. McKinsey have estimated that the value of analytics ranges from \$9.5 trillion to \$15.4 trillion while \$5 to 7 trillion can be attributed to the most advanced AI techniques.

Machine learning (ML) is the study of computer algorithms that improve automatically through experience. It is seen as a part of artificial intelligence. Machine learning algorithms build a model based on sample data, known as "training data", in order to make predictions or decisions without being explicitly programmed to do so. Machine learning algorithms are used in a wide variety of applications, such as email filtering and computer vision, where it is difficult or unfeasible to develop conventional algorithms to perform the needed tasks.

A subset of machine learning is closely related to computational statistics, which focuses on making predictions using computers; but not all machine learning is statistical learning. The study of mathematical optimization delivers methods, theory and application domains to the field of machine learning. Data mining is a related field of study, focusing on exploratory data analysis through unsupervised learning. In its application across business problems, machine learning is also referred to as predictive analytics.

## **Overview**

Machine learning involves computers discovering how they can perform tasks without being explicitly programmed to do so. It involves computers learning from data provided so that they carry out certain tasks. For simple tasks assigned to computers, it is possible to program algorithms telling the machine how to execute all steps required to solve the problem at hand; on the computer's part, no learning is needed. For more advanced tasks, it can be challenging for a human to manually create the needed algorithms. In practice, it can turn out to be more effective to help the machine develop its own algorithm, rather than having human programmers specify every needed step.

The discipline of machine learning employs various approaches to teach computers to accomplish tasks where no fully satisfactory algorithm is available. In cases where vast numbers of potential answers exist, one approach is to label some of the correct answers as valid. This can then be used as training data for the computer to improve the algorithm(s) it uses to determine correct answers. For example, to train a system for the task of digital character recognition, the MNIST dataset of handwritten digits has often been used.

## **Machine learning approaches**

Machine learning approaches are traditionally divided into three broad categories, depending on the nature of the "signal" or "feedback" available to the learning system:

**Supervised learning:** The computer is presented with example inputs and their desired outputs, given by a "teacher", and the goal is to learn a general rule that maps inputs to outputs.

**Unsupervised learning:** No labels are given to the learning algorithm, leaving it on its own to find structure in its input. Unsupervised learning can be a goal in itself (discovering hidden patterns in data) or a means towards an end (feature learning).

**Reinforcement learning:** A computer program interacts with a dynamic environment in which it must perform a certain goal (such as driving a vehicle or playing a game against an opponent). As it navigates its problem space, the program is provided feedback that's analogous to rewards, which it tries to maximize.

Other approaches have been developed which don't fit neatly into this three-fold categorisation, and sometimes more than one is used by the same machine learning system. For example topic modeling, dimensionality reduction or meta learning.

As of 2020, deep learning has become the dominant approach for much ongoing work in the field of machine learning.

## **History and relationships to other fields**

The term machine learning was coined in 1959 by Arthur Samuel, an American IBMer and pioneer in the field of computer gaming and artificial intelligence. A representative book of the machine learning research during the 1960s was the Nilsson's book on Learning Machines, dealing mostly with machine learning for pattern classification. Interest related to pattern recognition continued into the 1970s, as described by Duda and Hart in 1973. In 1981 a report was given on using

teaching strategies so that a neural network learns to recognize 40 characters (26 letters, 10 digits, and 4 special symbols) from a computer terminal.

Tom M. Mitchell provided a widely quoted, more formal definition of the algorithms studied in the machine learning field: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E." This definition of the tasks in which machine learning is concerned offers a fundamentally operational definition rather than defining the field in cognitive terms. This follows Alan Turing's proposal in his paper "Computing Machinery and Intelligence", in which the question "Can machines think?" is replaced with the question "Can machines do what we (as thinking entities) can do?".

Modern day machine learning has two objectives, one is to classify data based on models which have been developed, the other purpose is to make predictions for future outcomes based on these models. A hypothetical algorithm specific to classifying data may use computer vision of moles coupled with supervised learning in order to train it to classify the cancerous moles. Where as, a machine learning algorithm for stock trading may inform the trader of future potential predictions.

## **Artificial intelligence**

## Machine Learning as subfield of AI

Part of Machine Learning as subfield of AI or part of AI as subfield of Machine Learning

As a scientific endeavor, machine learning grew out of the quest for artificial intelligence. In the early days of AI as an academic discipline, some researchers were interested in having machines learn from data. They attempted to approach the problem with various symbolic methods, as well as what was then termed "neural networks"; these were mostly perceptrons and other models that were later found to be reinventions of the generalized linear models of statistics. Probabilistic reasoning was also employed, especially in automated medical diagnosis.

However, an increasing emphasis on the logical, knowledge-based approach caused a rift between AI and machine learning. Probabilistic systems were plagued by theoretical and practical problems of data acquisition and representation. By 1980, expert systems had come to dominate AI, and statistics was out of favor. Work on symbolic/knowledge-based learning did continue within AI, leading to inductive logic programming, but the more statistical line of research was now outside the field of AI proper, in pattern recognition and information retrieval. Neural networks research had been abandoned by AI and computer science around the same time. This line, too, was continued outside the AI/CS field, as "connectionism", by researchers from other disciplines including Hopfield, Rumelhart and Hinton. Their main success came in the mid-1980s with the reinvention of backpropagation.

Machine learning (ML), reorganized as a separate field, started to flourish in the 1990s. The field changed its goal from achieving artificial intelligence to tackling solvable problems of a practical nature. It shifted focus away from the symbolic

approaches it had inherited from AI, and toward methods and models borrowed from statistics and probability theory.

As of 2020, many sources continue to assert that machine learning remains a subfield of AI. The main disagreement is whether all of ML is part of AI, as this would mean that anyone using ML could claim they are using AI. Others have the view that not all of ML is part of AI where only an 'intelligent' subset of ML is part of AI.

The question to what is the difference between ML and AI is answered by Judea Pearl in The Book of Why. Accordingly ML learns and predicts based on passive observations, whereas AI implies an agent interacting with the environment to learn and take actions that maximize its chance of successfully achieving its goals.

## **Data mining**

Machine learning and data mining often employ the same methods and overlap significantly, but while machine learning focuses on prediction, based on known properties learned from the training data, data mining focuses on the discovery of (previously) unknown properties in the data (this is the analysis step of knowledge discovery in databases). Data mining uses many machine learning methods, but with different goals; on the other hand, machine learning also employs data mining methods as "unsupervised learning" or as a preprocessing step to improve learner accuracy. Much of the confusion between these two research communities (which do often have separate conferences and separate journals, ECML PKDD being a major exception) comes from the basic assumptions they work with: in machine learning, performance is usually evaluated with respect to the ability to reproduce known knowledge, while in knowledge discovery and data mining (KDD) the key

task is the discovery of previously unknown knowledge. Evaluated with respect to known knowledge, an uninformed (unsupervised) method will easily be outperformed by other supervised methods, while in a typical KDD task, supervised methods cannot be used due to the unavailability of training data.

## **Optimization**

Machine learning also has intimate ties to optimization: many learning problems are formulated as minimization of some loss function on a training set of examples. Loss functions express the discrepancy between the predictions of the model being trained and the actual problem instances (for example, in classification, one wants to assign a label to instances, and models are trained to correctly predict the pre-assigned labels of a set of examples).

## **Generalization**

The difference between optimization and machine learning arises from the goal of generalization: while optimization algorithms can minimize the loss on a training set, machine learning is concerned with minimizing the loss on unseen samples. Characterizing the generalization of various learning algorithms is an active topic of current research, especially for deep learning algorithms.

## **Statistics**

Machine learning and statistics are closely related fields in terms of methods, but distinct in their principal goal: statistics draws population inferences from a sample, while machine learning finds generalizable predictive patterns. According to Michael I. Jordan, the ideas of machine learning, from methodological principles

to theoretical tools, have had a long pre-history in statistics. He also suggested the term data science as a placeholder to call the overall field.

Leo Breiman distinguished two statistical modeling paradigms: data model and algorithmic model, wherein "algorithmic model" means more or less the machine learning algorithms like Random forest.

Some statisticians have adopted methods from machine learning, leading to a combined field that they call statistical learning.

## **Theory**

A core objective of a learner is to generalize from its experience. Generalization in this context is the ability of a learning machine to perform accurately on new, unseen examples/tasks after having experienced a learning data set. The training examples come from some generally unknown probability distribution (considered representative of the space of occurrences) and the learner has to build a general model about this space that enables it to produce sufficiently accurate predictions in new cases.

The computational analysis of machine learning algorithms and their performance is a branch of theoretical computer science known as computational learning theory. Because training sets are finite and the future is uncertain, learning theory usually does not yield guarantees of the performance of algorithms. Instead, probabilistic bounds on the performance are quite common. The bias–variance decomposition is one way to quantify generalization error.

For the best performance in the context of generalization, the complexity of the hypothesis should match the complexity of the function underlying the data. If the hypothesis is less complex than the function, then the model has under fitted the data. If the complexity of the model is increased in response, then the training error decreases. But if the hypothesis is too complex, then the model is subject to overfitting and generalization will be poorer.

In addition to performance bounds, learning theorists study the time complexity and feasibility of learning. In computational learning theory, a computation is considered feasible if it can be done in polynomial time. There are two kinds of time complexity results. Positive results show that a certain class of functions can be learned in polynomial time. Negative results show that certain classes cannot be learned in polynomial time.

## **Approaches**

### **Types of learning algorithms**

The types of machine learning algorithms differ in their approach, the type of data they input and output, and the type of task or problem that they are intended to solve.

### **Supervised learning**

A support vector machine is a supervised learning model that divides the data into regions separated by a linear boundary. Here, the linear boundary divides the black circles from the white.

Supervised learning algorithms build a mathematical model of a set of data that contains both the inputs and the desired outputs. The data is known as training data,

and consists of a set of training examples. Each training example has one or more inputs and the desired output, also known as a supervisory signal. In the mathematical model, each training example is represented by an array or vector, sometimes called a feature vector, and the training data is represented by a matrix. Through iterative optimization of an objective function, supervised learning algorithms learn a function that can be used to predict the output associated with new inputs. An optimal function will allow the algorithm to correctly determine the output for inputs that were not a part of the training data. An algorithm that improves the accuracy of its outputs or predictions over time is said to have learned to perform that task.

Types of supervised learning algorithms include active learning, classification and regression. Classification algorithms are used when the outputs are restricted to a limited set of values, and regression algorithms are used when the outputs may have any numerical value within a range. As an example, for a classification algorithm that filters emails, the input would be an incoming email, and the output would be the name of the folder in which to file the email.

Similarity learning is an area of supervised machine learning closely related to regression and classification, but the goal is to learn from examples using a similarity function that measures how similar or related two objects are. It has applications in ranking, recommendation systems, visual identity tracking, face verification, and speaker verification.

## **Unsupervised learning**

Unsupervised learning algorithms take a set of data that contains only inputs, and find structure in the data, like grouping or clustering of data points. The algorithms,

therefore, learn from test data that has not been labeled, classified or categorized. Instead of responding to feedback, unsupervised learning algorithms identify commonalities in the data and react based on the presence or absence of such commonalities in each new piece of data. A central application of unsupervised learning is in the field of density estimation in statistics, such as finding the probability density function. Though unsupervised learning encompasses other domains involving summarizing and explaining data features.

Cluster analysis is the assignment of a set of observations into subsets (called clusters) so that observations within the same cluster are similar according to one or more predesignated criteria, while observations drawn from different clusters are dissimilar. Different clustering techniques make different assumptions on the structure of the data, often defined by some similarity metric and evaluated, for example, by internal compactness, or the similarity between members of the same cluster, and separation, the difference between clusters. Other methods are based on estimated density and graph connectivity.

### **Semi-supervised learning**

Semi-supervised learning falls between unsupervised learning (without any labeled training data) and supervised learning (with completely labeled training data). Some of the training examples are missing training labels, yet many machine-learning researchers have found that unlabeled data, when used in conjunction with a small amount of labeled data, can produce a considerable improvement in learning accuracy.

In weakly supervised learning, the training labels are noisy, limited, or imprecise; however, these labels are often cheaper to obtain, resulting in larger effective training sets.

### **Reinforcement learning**

Reinforcement learning is an area of machine learning concerned with how software agents ought to take actions in an environment so as to maximize some notion of cumulative reward. Due to its generality, the field is studied in many other disciplines, such as game theory, control theory, operations research, information theory, simulation-based optimization, multi-agent systems, swarm intelligence, statistics and genetic algorithms. In machine learning, the environment is typically represented as a Markov decision process (MDP). Many reinforcement learning algorithms use dynamic programming techniques. Reinforcement learning algorithms do not assume knowledge of an exact mathematical model of the MDP, and are used when exact models are infeasible. Reinforcement learning algorithms are used in autonomous vehicles or in learning to play a game against a human opponent.

### **Self learning**

Self-learning as a machine learning paradigm was introduced in 1982 along with a neural network capable of self-learning named crossbar adaptive array (CAA). It is a learning with no external rewards and no external teacher advice. The CAA self-learning algorithm computes, in a crossbar fashion, both decisions about actions and emotions (feelings) about consequence situations. The system is driven by the interaction between cognition and emotion. The self-learning algorithm updates a memory matrix  $W = ||w(a,s)||$  such that in each iteration executes the following machine learning routine:

In situation  $s$  perform an action  $a$ ;  
Receive consequence situation  $s'$ ;  
Compute emotion of being in consequence situation  $v(s')$ ;  
Update crossbar memory  $w'(a,s) = w(a,s) + v(s')$ .

It is a system with only one input, situation  $s$ , and only one output, action (or behavior)  $a$ . There is neither a separate reinforcement input nor an advice input from the environment. The backpropagated value (secondary reinforcement) is the emotion toward the consequence situation. The CAA exists in two environments, one is the behavioral environment where it behaves, and the other is the genetic environment, wherefrom it initially and only once receives initial emotions about situations to be encountered in the behavioral environment. After receiving the genome (species) vector from the genetic environment, the CAA learns a goal-seeking behavior, in an environment that contains both desirable and undesirable situations.

## **Feature learning**

Several learning algorithms aim at discovering better representations of the inputs provided during training. Classic examples include principal components analysis and cluster analysis. Feature learning algorithms, also called representation learning algorithms, often attempt to preserve the information in their input but also transform it in a way that makes it useful, often as a pre-processing step before performing classification or predictions. This technique allows reconstruction of the inputs coming from the unknown data-generating distribution, while not being necessarily faithful to configurations that are implausible under that distribution. This replaces manual feature engineering, and allows a machine to both learn the features and use them to perform a specific task.

Feature learning can be either supervised or unsupervised. In supervised feature learning, features are learned using labeled input data. Examples include artificial neural networks, multilayer perceptrons, and supervised dictionary learning. In unsupervised feature learning, features are learned with unlabeled input data. Examples include dictionary learning, independent component analysis, autoencoders, matrix factorization and various forms of clustering.

Manifold learning algorithms attempt to do so under the constraint that the learned representation is low-dimensional. Sparse coding algorithms attempt to do so under the constraint that the learned representation is sparse, meaning that the mathematical model has many zeros. Multilinear subspace learning algorithms aim to learn low-dimensional representations directly from tensor representations for multidimensional data, without reshaping them into higher-dimensional vectors. Deep learning algorithms discover multiple levels of representation, or a hierarchy of features, with higher-level, more abstract features defined in terms of (or generating) lower-level features. It has been argued that an intelligent machine is one that learns a representation that disentangles the underlying factors of variation that explain the observed data.

Feature learning is motivated by the fact that machine learning tasks such as classification often require input that is mathematically and computationally convenient to process. However, real-world data such as images, video, and sensory data has not yielded to attempts to algorithmically define specific features. An alternative is to discover such features or representations thorough examination, without relying on explicit algorithms.

## **Sparse dictionary learning**

Sparse dictionary learning is a feature learning method where a training example is represented as a linear combination of basis functions, and is assumed to be a sparse matrix. The method is strongly NP-hard and difficult to solve approximately. A popular heuristic method for sparse dictionary learning is the K-SVD algorithm. Sparse dictionary learning has been applied in several contexts. In classification, the problem is to determine the class to which a previously unseen training example belongs. For a dictionary where each class has already been built, a new training example is associated with the class that is best sparsely represented by the corresponding dictionary. Sparse dictionary learning has also been applied in image de-noising. The key idea is that a clean image patch can be sparsely represented by an image dictionary, but the noise cannot.

## **Anomaly detection**

In data mining, anomaly detection, also known as outlier detection, is the identification of rare items, events or observations which raise suspicions by differing significantly from the majority of the data. Typically, the anomalous items represent an issue such as bank fraud, a structural defect, medical problems or errors in a text. Anomalies are referred to as outliers, novelties, noise, deviations and exceptions.

In particular, in the context of abuse and network intrusion detection, the interesting objects are often not rare objects, but unexpected bursts of inactivity. This pattern does not adhere to the common statistical definition of an outlier as a rare object, and many outlier detection methods (in particular, unsupervised algorithms) will fail on such data unless it has been aggregated appropriately.

Instead, a cluster analysis algorithm may be able to detect the micro-clusters formed by these patterns.

Three broad categories of anomaly detection techniques exist. Unsupervised anomaly detection techniques detect anomalies in an unlabeled test data set under the assumption that the majority of the instances in the data set are normal, by looking for instances that seem to fit least to the remainder of the data set. Supervised anomaly detection techniques require a data set that has been labeled as "normal" and "abnormal" and involves training a classifier (the key difference to many other statistical classification problems is the inherently unbalanced nature of outlier detection). Semi-supervised anomaly detection techniques construct a model representing normal behavior from a given normal training data set and then test the likelihood of a test instance to be generated by the model.

## **Robot learning**

In developmental robotics, robot learning algorithms generate their own sequences of learning experiences, also known as a curriculum, to cumulatively acquire new skills through self-guided exploration and social interaction with humans. These robots use guidance mechanisms such as active learning, maturation, motor synergies and imitation.

## **Association rules**

Association rule learning is a rule-based machine learning method for discovering relationships between variables in large databases. It is intended to identify strong rules discovered in databases using some measure of "interestingness".

Rule-based machine learning is a general term for any machine learning method that identifies, learns, or evolves "rules" to store, manipulate or apply knowledge. The defining characteristic of a rule-based machine learning algorithm is the identification and utilization of a set of relational rules that collectively represent the knowledge captured by the system. This is in contrast to other machine learning algorithms that commonly identify a singular model that can be universally applied to any instance in order to make a prediction. Rule-based machine learning approaches include learning classifier systems, association rule learning, and artificial immune systems.

Based on the concept of strong rules, Rakesh Agrawal, Tomasz Imieliński and Arun Swami introduced association rules for discovering regularities between products in large-scale transaction data recorded by point-of-sale (POS) systems in supermarkets. For example, the rule  $\{\mathrm{onions, potatoes}\} \rightarrow \{\mathrm{burger}\}$  found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, they are likely to also buy hamburger meat. Such information can be used as the basis for decisions about marketing activities such as promotional pricing or product placements. In addition to market basket analysis, association rules are employed today in application areas including Web usage mining, intrusion detection, continuous production, and bioinformatics. In contrast with sequence mining, association rule learning typically does not consider the order of items either within a transaction or across transactions.

Learning classifier systems (LCS) are a family of rule-based machine learning algorithms that combine a discovery component, typically a genetic algorithm,

with a learning component, performing either supervised learning, reinforcement learning, or unsupervised learning. They seek to identify a set of context-dependent rules that collectively store and apply knowledge in a piecewise manner in order to make predictions.

Inductive logic programming (ILP) is an approach to rule-learning using logic programming as a uniform representation for input examples, background knowledge, and hypotheses. Given an encoding of the known background knowledge and a set of examples represented as a logical database of facts, an ILP system will derive a hypothesized logic program that entails all positive and no negative examples. Inductive programming is a related field that considers any kind of programming language for representing hypotheses (and not only logic programming), such as functional programs.

Inductive logic programming is particularly useful in bioinformatics and natural language processing. Gordon Plotkin and Ehud Shapiro laid the initial theoretical foundation for inductive machine learning in a logical setting. Shapiro built their first implementation (Model Inference System) in 1981: a Prolog program that inductively inferred logic programs from positive and negative examples. The term inductive here refers to philosophical induction, suggesting a theory to explain observed facts, rather than mathematical induction, proving a property for all members of a well-ordered set.

## **Models**

Performing machine learning involves creating a model, which is trained on some training data and then can process additional data to make predictions. Various types of models have been used and researched for machine learning systems.

## **Artificial neural networks**

An artificial neural network is an interconnected group of nodes, akin to the vast network of neurons in a brain. Here, each circular node represents an artificial neuron and an arrow represents a connection from the output of one artificial neuron to the input of another.

Artificial neural networks (ANNs), or connectionist systems, are computing systems vaguely inspired by the biological neural networks that constitute animal brains. Such systems "learn" to perform tasks by considering examples, generally without being programmed with any task-specific rules.

An ANN is a model based on a collection of connected units or nodes called "artificial neurons", which loosely model the neurons in a biological brain. Each connection, like the synapses in a biological brain, can transmit information, a "signal", from one artificial neuron to another. An artificial neuron that receives a signal can process it and then signal additional artificial neurons connected to it. In common ANN implementations, the signal at a connection between artificial neurons is a real number, and the output of each artificial neuron is computed by some non-linear function of the sum of its inputs. The connections between artificial neurons are called "edges". Artificial neurons and edges typically have a weight that adjusts as learning proceeds. The weight increases or decreases the strength of the signal at a connection. Artificial neurons may have a threshold such that the signal is only sent if the aggregate signal crosses that threshold. Typically, artificial neurons are aggregated into layers. Different layers may perform different kinds of transformations on their inputs. Signals travel from the first layer (the

input layer) to the last layer (the output layer), possibly after traversing the layers multiple times.

The original goal of the ANN approach was to solve problems in the same way that a human brain would. However, over time, attention moved to performing specific tasks, leading to deviations from biology. Artificial neural networks have been used on a variety of tasks, including computer vision, speech recognition, machine translation, social network filtering, playing board and video games and medical diagnosis.

Deep learning consists of multiple hidden layers in an artificial neural network. This approach tries to model the way the human brain processes light and sound into vision and hearing. Some successful applications of deep learning are computer vision and speech recognition.

### **Decision trees**

Decision tree learning uses a decision tree as a predictive model to go from observations about an item (represented in the branches) to conclusions about the item's target value (represented in the leaves). It is one of the predictive modeling approaches used in statistics, data mining, and machine learning. Tree models where the target variable can take a discrete set of values are called classification trees; in these tree structures, leaves represent class labels and branches represent conjunctions of features that lead to those class labels. Decision trees where the target variable can take continuous values (typically real numbers) are called regression trees. In decision analysis, a decision tree can be used to visually and explicitly represent decisions and decision making. In data mining, a decision tree

describes data, but the resulting classification tree can be an input for decision making.

### **Support vector machines**

Support vector machines (SVMs), also known as support vector networks, are a set of related supervised learning methods used for classification and regression. Given a set of training examples, each marked as belonging to one of two categories, an SVM training algorithm builds a model that predicts whether a new example falls into one category or the other. An SVM training algorithm is a non-probabilistic, binary, linear classifier, although methods such as Platt scaling exist to use SVM in a probabilistic classification setting. In addition to performing linear classification, SVMs can efficiently perform a non-linear classification using what is called the kernel trick, implicitly mapping their inputs into high-dimensional feature spaces.

### **Regression analysis**

Regression analysis encompasses a large variety of statistical methods to estimate the relationship between input variables and their associated features. Its most common form is linear regression, where a single line is drawn to best fit the given data according to a mathematical criterion such as ordinary least squares. The latter is often extended by regularization (mathematics) methods to mitigate overfitting and bias, as in ridge regression. When dealing with non-linear problems, go-to models include polynomial regression (for example, used for trendline fitting in Microsoft Excel, logistic regression (often used in statistical classification) or even kernel regression, which introduces non-linearity by taking advantage of the kernel trick to implicitly map input variables to higher-dimensional space.

## **Bayesian networks**

A simple Bayesian network. Rain influences whether the sprinkler is activated, and both rain and the sprinkler influence whether the grass is wet.

A Bayesian network, belief network, or directed acyclic graphical model is a probabilistic graphical model that represents a set of random variables and their conditional independence with a directed acyclic graph (DAG). For example, a Bayesian network could represent the probabilistic relationships between diseases and symptoms. Given symptoms, the network can be used to compute the probabilities of the presence of various diseases. Efficient algorithms exist that perform inference and learning. Bayesian networks that model sequences of variables, like speech signals or protein sequences, are called dynamic Bayesian networks. Generalizations of Bayesian networks that can represent and solve decision problems under uncertainty are called influence diagrams.

## **Genetic algorithms**

A genetic algorithm (GA) is a search algorithm and heuristic technique that mimics the process of natural selection, using methods such as mutation and crossover to generate new genotypes in the hope of finding good solutions to a given problem. In machine learning, genetic algorithms were used in the 1980s and 1990s. Conversely, machine learning techniques have been used to improve the performance of genetic and evolutionary algorithms.

## **Training models**

Usually, machine learning models require a lot of data in order for them to perform well. Usually, when training a machine learning model, one needs to collect a large, representative sample of data from a training set. Data from the training set can be as varied as a corpus of text, a collection of images, and data collected from individual users of a service. Overfitting is something to watch out for when training a machine learning model. Trained models derived from biased data can result in skewed or undesired predictions. Algorithmic bias is a potential result from data not fully prepared for training.

### **Federated learning**

Federated learning is an adapted form of distributed artificial intelligence to training machine learning models that decentralizes the training process, allowing for users' privacy to be maintained by not needing to send their data to a centralized server. This also increases efficiency by decentralizing the training process to many devices. For example, Gboard uses federated machine learning to train search query prediction models on users' mobile phones without having to send individual searches back to Google

## **Deep Learning:**

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the back propagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

Machine-learning technology powers many aspects of modern society: from web searches to content filtering on social networks to recommendations on e-commerce websites, and it is increasingly present in consumer products such as cameras and smartphones. Machine-learning systems are used to identify objects in images, transcribe speech into text, match news items, posts or products with users' interests, and select relevant results of search. Increasingly, these applications make use of a class of techniques called deep learning. Conventional machine-learning techniques were limited in their ability to process natural data in their raw form. For decades, constructing a pattern-recognition or machine-learning system required careful engineering and considerable domain expertise to design a feature extractor that transformed the raw data (such as the pixel values of an image) into a suitable internal representation or feature vector from which the learning subsystem, often a classifier, could detect or classify patterns in the input. Representation learning is a set of methods that allows a machine to be fed with

raw data and to automatically discover the representations needed for detection or classification. Deep-learning methods are representation-learning methods with multiple levels of representation, obtained by composing simple but non-linear modules that each transform the representation at one level (starting with the raw input) into a representation at a higher, slightly more abstract level. With the composition of enough such transformations, very complex functions can be learned. For classification tasks, higher layers of representation amplify aspects of the input that are important for discrimination and suppress irrelevant variations. An image, for example, comes in the form of an array of pixel values, and the learned features in the first layer of representation typically represent the presence or absence of edges at particular orientations and locations in the image. The second layer typically detects motifs by spotting particular arrangements of edges, regardless of small variations in the edge positions. The third layer may assemble motifs into larger combinations that correspond to parts of familiar objects, and subsequent layers would detect objects as combinations of these parts. The key aspect of deep learning is that these layers of features are not designed by human engineers: they are learned from data using a general-purpose learning procedure. Deep learning is making major advances in solving problems that have resisted the best attempts of the artificial intelligence community for many years. It has turned out to be very good at discovering intricate structures in high-dimensional data and is therefore applicable to many domains of science, business and government. In addition to beating records in image recognition and speech recognition, it has beaten other machine-learning techniques at predicting the activity of potential drug molecules, analysing particle accelerator data, reconstructing brain circuits, and predicting the effects of mutations in non-coding DNA on gene expression and disease. Perhaps more surprisingly, deep learning has produced extremely promising results for various tasks in natural language understanding, particularly

topic classification, sentiment analysis, question answering and language translation. We think that deep learning will have many more successes in the near future because it requires very little engineering by hand, so it can easily take advantage of increases in the amount of available computation and data. New learning algorithms and architectures that are currently being developed for deep neural networks will only accelerate this progress.

### **Supervised learning**

The most common form of machine learning, deep or not, is supervised learning. Imagine that we want to build a system that can classify images as containing, say, a house, a car, a person or a pet. We first collect a large data set of images of houses, cars, people and pets, each labelled with its category. During training, the machine is shown an image and produces an output in the form of a vector of scores, one for each category. We want the desired category to have the highest score of all categories, but this is unlikely to happen before training. We compute an objective function that measures the error (or distance) between the output scores and the desired pattern of scores. The machine then modifies its internal adjustable parameters to reduce this error. These adjustable parameters, often called weights, are real numbers that can be seen as ‘knobs’ that define the input–output function of the machine. In a typical deep-learning system, there may be hundreds of millions of these adjustable weights, and hundreds of millions of labelled examples with which to train the machine. To properly adjust the weight vector, the learning algorithm computes a gradient vector that, for each weight, indicates by what amount the error would increase or decrease if the weight were increased by a tiny amount. The weight vector is then adjusted in the opposite direction to the gradient vector. The objective function, averaged over all the training examples, can be seen as a kind of hilly landscape in the high-dimensional

space of weight values. The negative gradient vector indicates the direction of steepest descent in this landscape, taking it closer to a minimum, where the output error is low on average. In practice, most practitioners use a procedure called stochastic gradient descent (SGD). This consists of showing the input vector for a few examples, computing the outputs and the errors, computing the average gradient for those examples, and adjusting the weights accordingly. The process is repeated for many small sets of examples from the training set until the average of the objective function stops decreasing. It is called stochastic because each small set of examples gives a noisy estimate of the average gradient over all examples. This simple procedure usually finds a good set of weights surprisingly quickly when compared with far more elaborate optimization techniques. After training, the performance of the system is measured on a different set of examples called a test set. This serves to test the generalization ability of the machine — its ability to produce sensible answers on new inputs that it has never seen during training.

Many of the current practical applications of machine learning use linear classifiers on top of hand-engineered features. A two-class linear classifier computes a weighted sum of the feature vector components. If the weighted sum is above a threshold, the input is classified as belonging to a particular category. Since the 1960s we have known that linear classifiers can only carve their input space into very simple regions, namely half-spaces separated by a hyperplane. But problems such as image and speech recognition require the input–output function to be insensitive to irrelevant variations of the input, such as variations in position, orientation or illumination of an object, or variations in the pitch or accent of speech, while being very sensitive to particular minute variations (for example, the difference between a white wolf and a breed of wolf-like white dog called a Samoyed). At the pixel level, images of two Samoyeds in different poses and in

different environments may be very different from each other, whereas two images of a Samoyed and a wolf in the same position and on similar backgrounds may be very similar to each other. A linear classifier, or any other ‘shallow’ classifier operating on raw pixels could not possibly distinguish the latter two, while putting the former two in the same category. This is why shallow classifiers require a good feature extractor that solves the selectivity–invariance dilemma — one that produces representations that are selective to the aspects of the image that are important for discrimination, but that are invariant to irrelevant aspects such as the pose of the animal. To make classifiers more powerful, one can use generic non-linear features, as with kernel methods, but generic features such as those arising with the Gaussian kernel do not allow the learner to generalize well far from the training examples. The conventional option is to hand design good feature extractors, which requires a considerable amount of engineering skill and domain expertise. But this can all be avoided if good features can be learned automatically using a general-purpose learning procedure. This is the key advantage of deep learning. A deep-learning architecture is a multilayer stack of simple modules, all (or most) of which are subject to learning, and many of which compute non-linear input–output mappings. Each module in the stack transforms its input to increase both the selectivity and the invariance of the representation. With multiple non-linear layers, say a depth of 5 to 20, a system can implement extremely intricate functions of its inputs that are simultaneously sensitive to minute details — distinguishing Samoyeds from white wolves — and insensitive to large irrelevant variations such as the background, pose, lighting and surrounding objects.

### **Backpropagation to train multilayer architectures**

From the earliest days of pattern recognition, the aim of researchers has been to replace hand-engineered features with trainable multilayer networks, but despite its

simplicity, the solution was not widely understood until the mid 1980s. As it turns out, multilayer architectures can be trained by simple stochastic gradient descent. As long as the modules are relatively smooth functions of their inputs and of their internal weights, one can compute gradients using the backpropagation procedure. The idea that this could be done, and that it worked, was discovered independently by several different groups during the 1970s and 1980s. The backpropagation procedure to compute the gradient of an objective function with respect to the weights of a multilayer stack of modules is nothing more than a practical application of the chain rule for derivatives. The key insight is that the derivative (or gradient) of the objective with respect to the input of a module can be computed by working backwards from the gradient with respect to the output of that module (or the input of the subsequent module). The back propagation equation can be applied repeatedly to propagate gradients through all modules, starting from the output at the top (where the network produces its prediction) all the way to the bottom (where the external input is fed). Once these gradients have been computed, it is straightforward to compute the gradients with respect to the weights of each module. Many applications of deep learning use feedforward neural network architectures (Fig. 1), which learn to map a fixed-size input (for example, an image) to a fixed-size output (for example, a probability for each of several categories). To go from one layer to the next, a set of units compute a weighted sum of their inputs from the previous layer and pass the result through a non-linear function. At present, the most popular non-linear function is the rectified linear unit (ReLU), which is simply the half-wave rectifier  $f(z) = \max(z, 0)$ . In past decades, neural nets used smoother non-linearities, such as  $\tanh(z)$  or  $1/(1+\exp(-z))$ , but the ReLU typically learns much faster in networks with many layers, allowing training of a deep supervised network without unsupervised pre-training. Units that are not in the input or output layer are conventionally called

hidden units. The hidden layers can be seen as distorting the input in a non-linear way so that categories become linearly separable by the last layer (Fig. 1). In the late 1990s, neural nets and backpropagation were largely forsaken by the machine-learning community and ignored by the computer-vision and speech-recognition communities. It was widely thought that learning useful, multistage, feature extractors with little prior knowledge was infeasible. In particular, it was commonly thought that simple gradient descent would get trapped in poor local minima — weight configurations for which no small change would reduce the average error. In practice, poor local minima are rarely a problem with large networks. Regardless of the initial conditions, the system nearly always reaches solutions of very similar quality. Recent theoretical and empirical results strongly suggest that local minima are not a serious issue in general. Instead, the landscape is packed with a combinatorially large number of saddle points where the gradient is zero, and the surface curves up in most dimensions and curves down in the remainder. The analysis seems to show that saddle points with only a few downward curving directions are present in very large numbers, but almost all of them have very similar values of the objective function. Hence, it does not much matter which of these saddle points the algorithm gets stuck at. Interest in deep feed forward networks was revived around 2006 by a group of researchers brought together by the Canadian Institute for Advanced Research (CIFAR). The researchers introduced unsupervised learning procedures that could create layers of feature detectors without requiring labelled data. The objective in learning each layer of feature detectors was to be able to reconstruct or model the activities of feature detectors (or raw inputs) in the layer below. By ‘pre-training’ several layers of progressively more complex feature detectors using this reconstruction objective, the weights of a deep network could be initialized to sensible values. A final layer of output units could then be added to the top of the network and the whole deep

system could be fine-tuned using standard backpropagation. This worked remarkably well for recognizing handwritten digits or for detecting pedestrians, especially when the amount of labelled data was very limited. The first major application of this pre-training approach was in speech recognition, and it was made possible by the advent of fast graphics processing units (GPUs) that were convenient to program and allowed researchers to train networks 10 or 20 times faster. In 2009, the approach was used to map short temporal windows of coefficients extracted from a sound wave to a set of probabilities for the various fragments of speech that might be represented by the frame in the centre of the window. It achieved record-breaking results on a standard speech recognition benchmark that used a small vocabulary<sup>38</sup> and was quickly developed to give record-breaking results on a large vocabulary task<sup>39</sup>. By 2012, versions of the deep net from 2009 were being developed by many of the major speech groups<sup>6</sup> and were already being deployed in Android phones. For smaller data sets, unsupervised pre-training helps to prevent overfitting, leading to significantly better generalization when the number of labelled examples is small, or in a transfer setting where we have lots of examples for some ‘source’ tasks but very few for some ‘target’ tasks. Once deep learning had been rehabilitated, it turned out that the pre-training stage was only needed for small data sets. There was, however, one particular type of deep, feedforward network that was much easier to train and generalized much better than networks with full connectivity between adjacent layers. This was the convolutional neural network (ConvNet). It achieved many practical successes during the period when neural networks were out of favour and it has recently been widely adopted by the computer vision community.

## **Convolutional neural networks**

ConvNets are designed to process data that come in the form of multiple arrays, for example a colour image composed of three 2D arrays containing pixel intensities in the three colour channels. Many data modalities are in the form of multiple arrays: 1D for signals and sequences, including language; 2D for images or audio spectrograms; and 3D for video or volumetric images. There are four key ideas behind ConvNets that take advantage of the properties of natural signals: local connections, shared weights, pooling and the use of many layers. The architecture of a typical ConvNet is structured as a series of stages. The first few stages are composed of two types of layers: convolutional layers and pooling layers. Units in a convolutional layer are organized in feature maps, within which each unit is connected to local patches in the feature maps of the previous layer through a set of weights called a filter bank. The result of this local weighted sum is then passed through a non-linearity such as a ReLU. All units in a feature map share the same filter bank. Different feature maps in a layer use different filter banks. The reason for this architecture is twofold. First, in array data such as images, local groups of values are often highly correlated, forming distinctive local motifs that are easily detected. Second, the local statistics of images and other signals are invariant to location. In other words, if a motif can appear in one part of the image, it could appear anywhere, hence the idea of units at different locations sharing the same weights and detecting the same pattern in different parts of the array. Mathematically, the filtering operation performed by a feature map is a discrete convolution, hence the name. Although the role of the convolutional layer is to detect local conjunctions of features from the previous layer, the role of the pooling layer is to merge semantically similar features into one. Because the relative positions of the features forming a motif can vary somewhat, reliably detecting the motif can be done by coarse-graining the position of each feature. A typical pooling unit computes the maximum of a local patch of units in one feature

map (or in a few feature maps). Neighbouring pooling units take input from patches that are shifted by more than one row or column, thereby reducing the dimension of the representation and creating an invariance to small shifts and distortions. Two or three stages of convolution, non-linearity and pooling are stacked, followed by more convolutional and fully-connected layers. Backpropagating gradients through a ConvNet is as simple as through a regular deep network, allowing all the weights in all the filter banks to be trained. Deep neural networks exploit the property that many natural signals are compositional hierarchies, in which higher-level features are obtained by composing lower-level ones. In images, local combinations of edges form motifs, motifs assemble into parts, and parts form objects. Similar hierarchies exist in speech and text from sounds to phones, phonemes, syllables, words and sentences. The pooling allows representations to vary very little when elements in the previous layer vary in position and appearance. The convolutional and pooling layers in ConvNets are directly inspired by the classic notions of simple cells and complex cells in visual neuroscience, and the overall architecture is reminiscent of the LGN–V1–V2–V4–IT hierarchy in the visual cortex ventral pathway. When ConvNet models and monkeys are shown the same picture, the activations of high-level units in the ConvNet explains half of the variance of random sets of 160 neurons in the monkey’s inferotemporal cortex. ConvNets have their roots in the neocognitron<sup>46</sup>, the architecture of which was somewhat similar, but did not have an end-to-end supervised-learning algorithm such as backpropagation. A primitive 1D ConvNet called a time-delay neural net was used for the recognition of phonemes and simple words. There have been numerous applications of convolutional networks going back to the early 1990s, starting with time-delay neural networks for speech recognition and document reading. The document reading system used a ConvNet trained jointly with a probabilistic model that implemented language constraints.

By the late 1990s this system was reading over 10% of all the cheques in the United States. A number of ConvNet-based optical character recognition and handwriting recognition systems were later deployed by Microsoft<sup>49</sup>. ConvNets were also experimented with in the early 1990s for object detection in natural images, including faces and hands, and for face recognition.

### **Image understanding with deep convolutional networks**

Since the early 2000s, ConvNets have been applied with great success to the detection, segmentation and recognition of objects and regions in images. These were all tasks in which labelled data was relatively abundant, such as traffic sign recognition, the segmentation of biological images particularly for connectomics, and the detection of faces, text, pedestrians and human bodies in natural images. A major recent practical success of ConvNets is face recognition. Importantly, images can be labelled at the pixel level, which will have applications in technology, including autonomous mobile robots and self-driving cars. Companies such as Mobileye and NVIDIA are using such ConvNet-based methods in their upcoming vision systems for cars. Other applications gaining importance involve natural language understanding and speech recognition . Despite these successes, ConvNets were largely forsaken by the mainstream computer-vision and machine-learning communities until the ImageNet competition in 2012. When deep convolutional networks were applied to a data set of about a million images from the web that contained 1,000 different classes, they achieved spectacular results, almost halving the error rates of the best competing approaches. This success came from the efficient use of GPUs, ReLUs, a new regularization technique called dropout, and techniques to generate more training examples by deforming the existing ones. This success has brought about a revolution in computer vision;

ConvNets are now the dominant approach for almost all recognition and detection tasks and approach human performance on some tasks. A recent stunning demonstration combines ConvNets and recurrent net modules for the generation of image captions. Recent ConvNet architectures have 10 to 20 layers of ReLUs, hundreds of millions of weights, and billions of connections between units. Whereas training such large networks could have taken weeks only two years ago, progress in hardware, software and algorithm parallelization have reduced training times to a few hours. The performance of ConvNet-based vision systems has caused most major technology companies, including Google, Facebook, Microsoft, IBM, Yahoo!, Twitter and Adobe, as well as a quickly growing number of start-ups to initiate research and development projects and to deploy ConvNet-based image understanding products and services. ConvNets are easily amenable to efficient hardware implementations in chips or field-programmable gate arrays. A number of companies such as NVIDIA, Mobileye, Intel, Qualcomm and Samsung are developing ConvNet chips to enable real-time vision applications in smartphones, cameras, robots and self-driving cars. Distributed representations and language processing Deep-learning theory shows that deep nets have two different exponential advantages over classic learning algorithms that do not use distributed representations. Both of these advantages arise from the power of composition and depend on the underlying data-generating distribution having an appropriate componential structure. First, learning distributed representations enable generalization to new combinations of the values of learned features beyond those seen during training (for example,  $2^n$  combinations are possible with  $n$  binary features). Second, composing layers of representation in a deep net brings the potential for another exponential advantage (exponential in the depth). The hidden layers of a multilayer neural network learn to represent the network's inputs in a way that makes it easy to predict the target outputs. This is nicely demonstrated by

training a multilayer neural network to predict the next word in a sequence from a local context of earlier words. Each word in the context is presented to the network as a one-of-N vector, that is, one component has a value of 1 and the rest are 0. In the first layer, each word creates a different pattern of activations, or word vectors. In a language model, the other layers of the network learn to convert the input word vectors into an output word vector for the predicted next word, which can be used to predict the probability for any word in the vocabulary to appear as the next word. The network learns word vectors that contain many active components each of which can be interpreted as a separate feature of the word, as was first demonstrated in the context of learning distributed representations for symbols. These semantic features were not explicitly present in the input. They were discovered by the learning procedure as a good way of factorizing the structured relationships between the input and output symbols into multiple ‘micro-rules’. Learning word vectors turned out to also work very well when the word sequences come from a large corpus of real text and the individual micro-rules are unreliable. When trained to predict the next word in a news story, for example, the learned word vectors for Tuesday and Wednesday are very similar, as are the word vectors for Sweden and Norway. Such representations are called distributed representations because their elements (the features) are not mutually exclusive and their many configurations correspond to the variations seen in the observed data. These word vectors are composed of learned features that were not determined ahead of time by experts, but automatically discovered by the neural network. Vector representations of words learned from text are now very widely used in natural language applications. The issue of representation lies at the heart of the debate between the logic-inspired and the neural-network-inspired paradigms for cognition. In the logic-inspired paradigm, an instance of a symbol is something for which the only property is that it is either identical or non-identical

to other symbol instances. It has no internal structure that is relevant to its use; and to reason with symbols, they must be bound to the variables in judiciously chosen rules of inference. By contrast, neural networks just use big activity vectors, big weight matrices and scalar non-linearities to perform the type of fast ‘intuitive’ inference that underpins effortless commonsense reasoning. Before the introduction of neural language models, the standard approach to statistical modelling of language did not exploit distributed representations: it was based on counting frequencies of occurrences of short symbol sequences of length up to  $N$  (called  $N$ -grams). The number of possible  $N$ -grams is on the order of  $V^N$ , where  $V$  is the vocabulary size, so taking into account a context of more than a handful of words would require very large training corpora.  $N$ -grams treat each word as an atomic unit, so they cannot generalize across semantically related sequences of words, whereas neural language models can because they associate each word with a vector of real valued features, and semantically related words end up close to each other in that vector space.

## **Recurrent neural networks**

When backpropagation was first introduced, its most exciting use was for training recurrent neural networks (RNNs). For tasks that involve sequential inputs, such as speech and language, it is often better to use RNNs. RNNs process an input sequence one element at a time, maintaining in their hidden units a ‘state vector’ that implicitly contains information about the history of all the past elements of the sequence. When we consider the outputs of the hidden units at different discrete time steps as if they were the outputs of different neurons in a deep multilayer network, it becomes clear how we can apply backpropagation to train RNNs. RNNs are very powerful dynamic systems, but training them has proved to be

problematic because the backpropagated gradients either grow or shrink at each time step, so over many time steps they typically explode or vanish. Thanks to advances in their architecture and ways of training them, RNNs have been found to be very good at predicting the next character in the text<sup>83</sup> or the next word in a sequence, but they can also be used for more complex tasks. For example, after reading an English sentence one word at a time, an English ‘encoder’ network can be trained so that the final state vector of its hidden units is a good representation of the thought expressed by the sentence. This thought vector can then be used as the initial hidden state of (or as extra input to) a jointly trained French ‘decoder’ network, which outputs a probability distribution for the first word of the French translation. If a particular first word is chosen from this distribution and provided as input to the decoder network it will then output a probability distribution for the second word of the translation and so on until a full stop is chosen. Overall, this process generates sequences of French words according to a probability distribution that depends on the English sentence. This rather naive way of performing machine translation has quickly become competitive with the state-of-the-art, and this raises serious doubts about whether understanding a sentence requires anything like the internal symbolic expressions that are manipulated by using inference rules. It is more compatible with the view that everyday reasoning involves many simultaneous analogies that each contribute plausibility to a conclusion. Instead of translating the meaning of a French sentence into an English sentence, one can learn to ‘translate’ the meaning of an image into an English sentence. The encoder here is a deep ConvNet that converts the pixels into an activity vector in its last hidden layer. The decoder is an RNN similar to the ones used for machine translation and neural language modelling. There has been a surge of interest in such systems recently. RNNs, once unfolded in time (Fig. 5), can be seen as very deep feedforward networks in which all the layers share the

same weights. Although their main purpose is to learn long-term dependencies, theoretical and empirical evidence shows that it is difficult to learn to store information for very long. To correct for that, one idea is to augment the network with an explicit memory. The first proposal of this kind is the long short-term memory (LSTM) networks that use special hidden units, the natural behaviour of which is to remember inputs for a long time. A special unit called the memory cell acts like an accumulator or a gated leaky neuron: it has a connection to itself at the next time step that has a weight of one, so it copies its own real-valued state and accumulates the external signal, but this self-connection is multiplicatively gated by another unit that learns to decide when to clear the content of the memory. LSTM networks have subsequently proved to be more effective than conventional RNNs, especially when they have several layers for each time step, enabling an entire speech recognition system that goes all the way from acoustics to the sequence of characters in the transcription. LSTM networks or related forms of gated units are also currently used for the encoder and decoder networks that perform so well at machine translation. Over the past year, several authors have made different proposals to augment RNNs with a memory module. Proposals include the Neural Turing Machine in which the network is augmented by a ‘tape-like’ memory that the RNN can choose to read from or write to<sup>88</sup>, and memory networks, in which a regular network is augmented by a kind of associative memory<sup>89</sup>. Memory networks have yielded excellent performance on standard question-answering benchmarks. The memory is used to remember the story about which the network is later asked to answer questions. Beyond simple memorization, neural Turing machines and memory networks are being used for tasks that would normally require reasoning and symbol manipulation. Neural Turing machines can be taught ‘algorithms’. Among other things, they can learn to output a sorted list of symbols when their input consists of an unsorted sequence in which each symbol is

accompanied by a real value that indicates its priority in the list. Memory networks can be trained to keep track of the state of the world in a setting similar to a text adventure game and after reading a story, they can answer questions that require complex inference. In one test example, the network is shown a 15-sentence version of the The Lord of the Rings and correctly answers questions such as “where is Frodo now?”.

## **The future of deep learning**

Unsupervised learning had a catalytic effect in reviving interest in deep learning, but has since been overshadowed by the successes of purely supervised learning. Although we have not focused on it in this Review, we expect unsupervised learning to become far more important in the longer term. Human and animal learning is largely unsupervised: we discover the structure of the world by observing it, not by being told the name of every object. Human vision is an active process that sequentially samples the optic array in an intelligent, task-specific way using a small, high-resolution fovea with a large, low-resolution surround. We expect much of the future progress in vision to come from systems that are trained end-to-end and combine ConvNets with RNNs that use reinforcement learning to decide where to look. Systems combining deep learning and reinforcement learning are in their infancy, but they already outperform passive vision systems at classification tasks and produce impressive results in learning to play many different video games. Natural language understanding is another area in which deep learning is poised to make a large impact over the next few years. We expect systems that use RNNs to understand sentences or whole documents will become much better when they learn strategies for selectively attending to one part at a time. Ultimately, major progress in artificial intelligence will come about through

systems that combine representation learning with complex reasoning. Although deep learning and simple reasoning have been used for speech and handwriting recognition for a long time, new paradigms are needed to replace rule-based manipulation of symbolic expressions by operations on large vectors.

## **LITERATURE SURVEY**

### **1) Stress and health: Psychological, behavioral, and biological determinants**

**AUTHORS:** N. Schneiderman, G. Ironson, and S. D. Siegel

Stressors have a major influence upon mood, our sense of well-being, behavior, and health. Acute stress responses in young, healthy individuals may be adaptive and typically do not impose a health burden. However, if the threat is unremitting, particularly in older or unhealthy individuals, the long-term effects of stressors can damage health. The relationship between psychosocial stressors and disease is affected by the nature, number, and persistence of the stressors as well as by the individual's biological vulnerability (i.e., genetics, constitutional factors), psychosocial resources, and learned patterns of coping. Psychosocial interventions have proven useful for treating stress-related disorders and may influence the course of chronic diseases.

## **2) Sleep, Sleep Disorders, and Stress**

**AUTHORS:** A. N. Vgontzas, S. Pejovic, and M. Karataraki

The purpose of this study was to review potential, physiological, hormonal and neuronal mechanisms that may mediate the sleep changes. This paper investigates the literatures regarding the activity of the hypothalamic-pituitary-adrenal (HPA) axis, one of the main neuroendocrine stress systems during sleep in order to identify relations between stress and sleep disorder and the treatment of stress-induced insomnia. Sleep and wakefulness are regulated by the aminergic, cholinergic brainstem and hypothalamic systems. Activation of the HPA and/or the sympathetic nervous systems results in wakefulness and these hormones including corticotropin-releasing hormone (CRH), adrenocorticotrophic hormone (ACTH), cortisol or corticosterone, noradrenaline, and adrenaline, are associated with attention and arousal. Stress-related insomnia leads to a vicious circle by activating the HPA system. An awareness of the close interaction between sleep and stress systems is emerging and the hypothalamus is now recognized as a key center for sleep regulation, with hypothalamic neurotransmitter systems providing the framework for therapeutic advances. An updated understanding of these systems may allow researchers to elucidate neural mechanisms of sleep disorder and to develop effective intervention for sleep disorder.

### **3) A study of learning stress and stress management strategies of the students of postgraduate level: A case study of Islamia university of Bahawalpur, Pakistan**

**AUTHORS:** Q. Bukhsh, A. Shahzad, and M. Nisa

This study was designed to study Learning Stress and Stress Management Strategies for University Students. The objectives of the study were to find out the common symptoms of stress, causes of stress and to suggest stress management strategies used by students for coping with stress. For collecting data, survey method was adopted. This study was completed by following the procedure of descriptive method of research. 39 questions were used for data collection. 200 students from different departments of The Islamia University of Bahawalpur were chosen as a sample of the study. This study concluded that majority of students felt fatigue during stress. Burden of study, assignments were major causes of stress and they agreed that they reduce stress by watching T.V/Movies, listen to music or take part in other leisure time activities. It was recommended that burden of study may be reduced by conducting co curricular activities in academic area

#### **4) Personalized sleep parameters estimation from actigraphy: A machine learning approach**

**AUTHORS:** A. Khademi, Y. El-Manzalawy, L. Master, O. M. Buxton, and V. G. Honavar

The current gold standard for measuring sleep is polysomnography (PSG), but it can be obtrusive and costly. Actigraphy is a relatively low-cost and unobtrusive alternative to PSG. Of particular interest in measuring sleep from actigraphy is prediction of sleep-wake states. Current literature on prediction of sleep-wake states from actigraphy consists of methods that use population data, which we call generalized models. However, accounting for variability of sleep patterns across individuals calls for personalized models of sleep-wake states prediction that could be potentially better suited to individual-level data and yield more accurate estimation of sleep.

##### **Purpose**

To investigate the validity of developing personalized machine learning models, trained and tested on individual-level actigraphy data, for improved prediction of sleep-wake states and reliable estimation of nightly sleep parameters.

## **5) SaYoPillow: Blockchain-Integrated Privacy-Assured IoMT Framework for Stress Management Considering Sleeping Habits**

**AUTHORS:** L. Rachakonda, A. K. Bapatla, S. P. Mohanty, and E. Kougianos

Considering today's lifestyle, people just sleep forgetting the benefits sleep provides to the human body. Smart-Yoga Pillow (SaYoPillow) is proposed to help in understanding the relationship between stress and sleep and to fully materialize the idea of “Smart-Sleeping” by proposing an edge device. An edge processor with a model analyzing the physiological changes that occur during sleep along with the sleeping habits is proposed. Based on these changes during sleep, stress prediction for the following day is proposed. The secure transfer of the analyzed stress data along with the average physiological changes to the IoT cloud for storage is implemented. A secure transfer of any data from the cloud to any third party applications is also proposed. A user interface is provided allowing the user to control the data accessibility and visibility. SaYoPillow is novel, with security features as well as consideration of sleeping habits for stress reduction, with an accuracy of up to 96%.

## **SYSTEM STUDY**

### **FEASIBILITY STUDY**

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

### **ECONOMICAL FEASIBILITY**

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was

achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

## **TECHNICAL FEASIBILITY**

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

## **SOCIAL FEASIBILITY**

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

## **SYSTEM REQUIREMENTS**

### **HARDWARE REQUIREMENTS:**

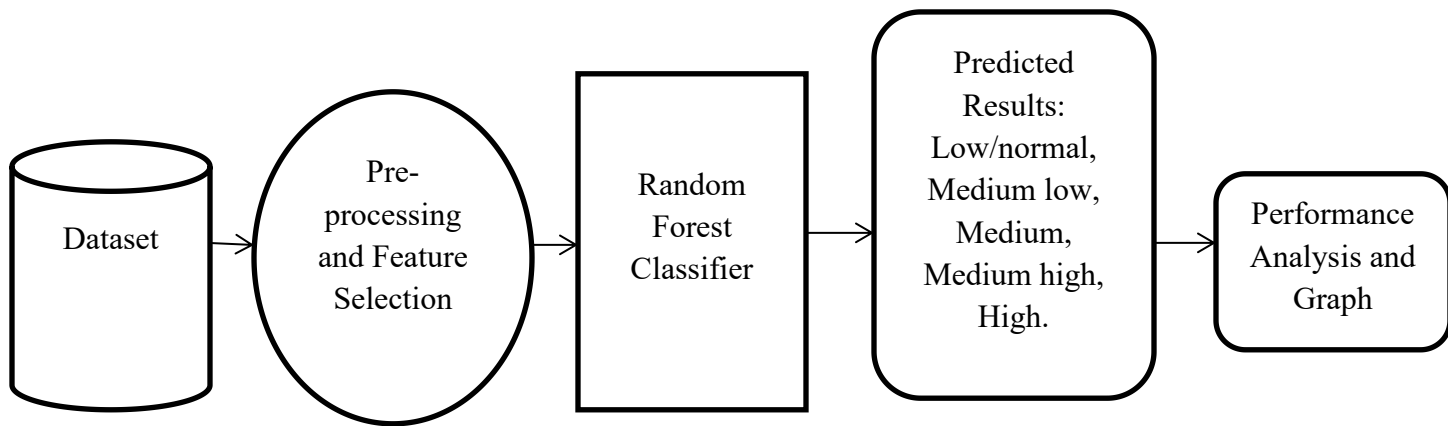
- System : Pentium i3 Processor.
- Hard Disk : 500 GB.
- Monitor : 15'' LED
- Input Devices : Keyboard, Mouse
- Ram : 4 GB

### **SOFTWARE REQUIREMENTS:**

- Operating system : Windows 10.
- Coding Language : Python.
- Web Framework : Flask.

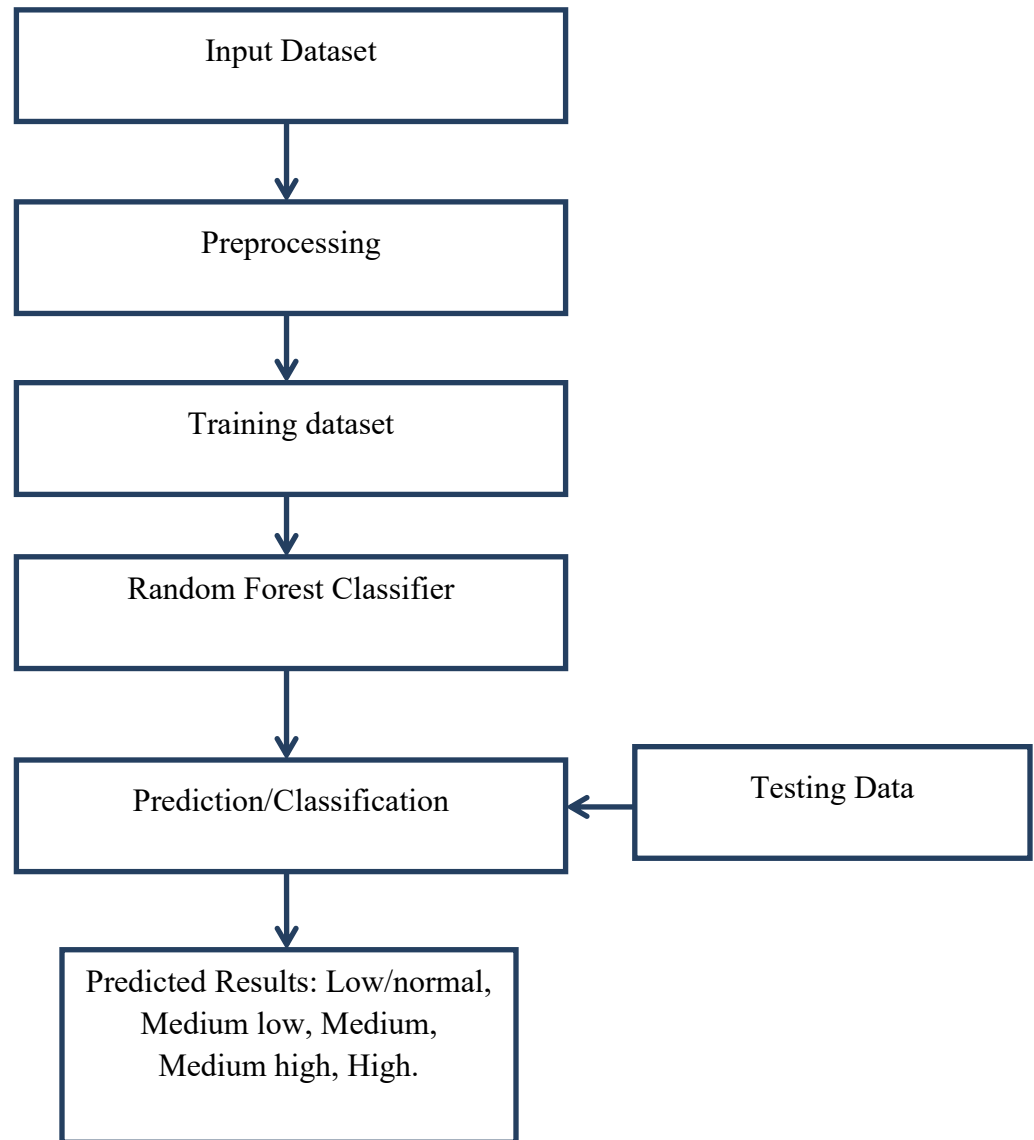
## SYSTEM DESIGN

### SYSTEM ARCHITECTURE:



## **DATA FLOW DIAGRAM:**

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.



## **UML DIAGRAMS**

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

### **GOALS:**

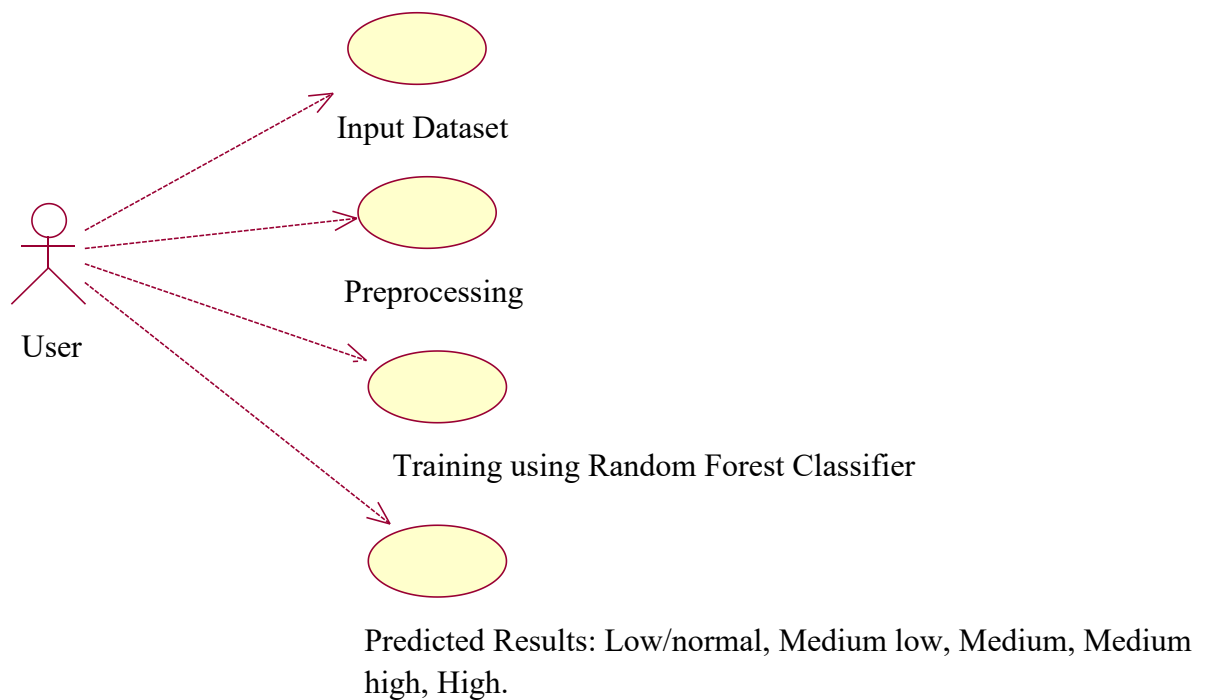
The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.

3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

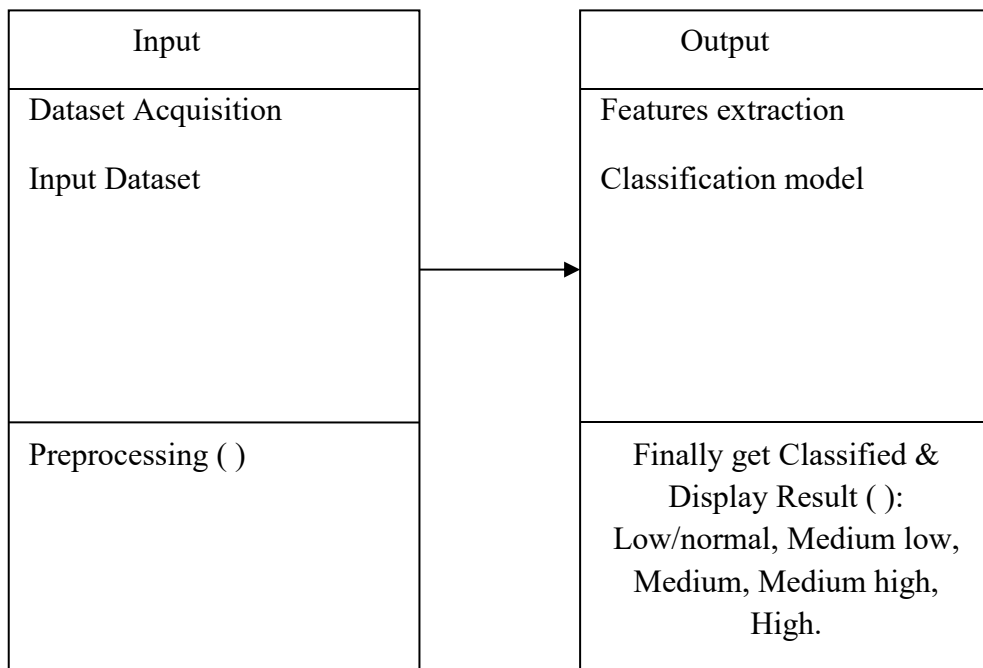
## **USE CASE DIAGRAM:**

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



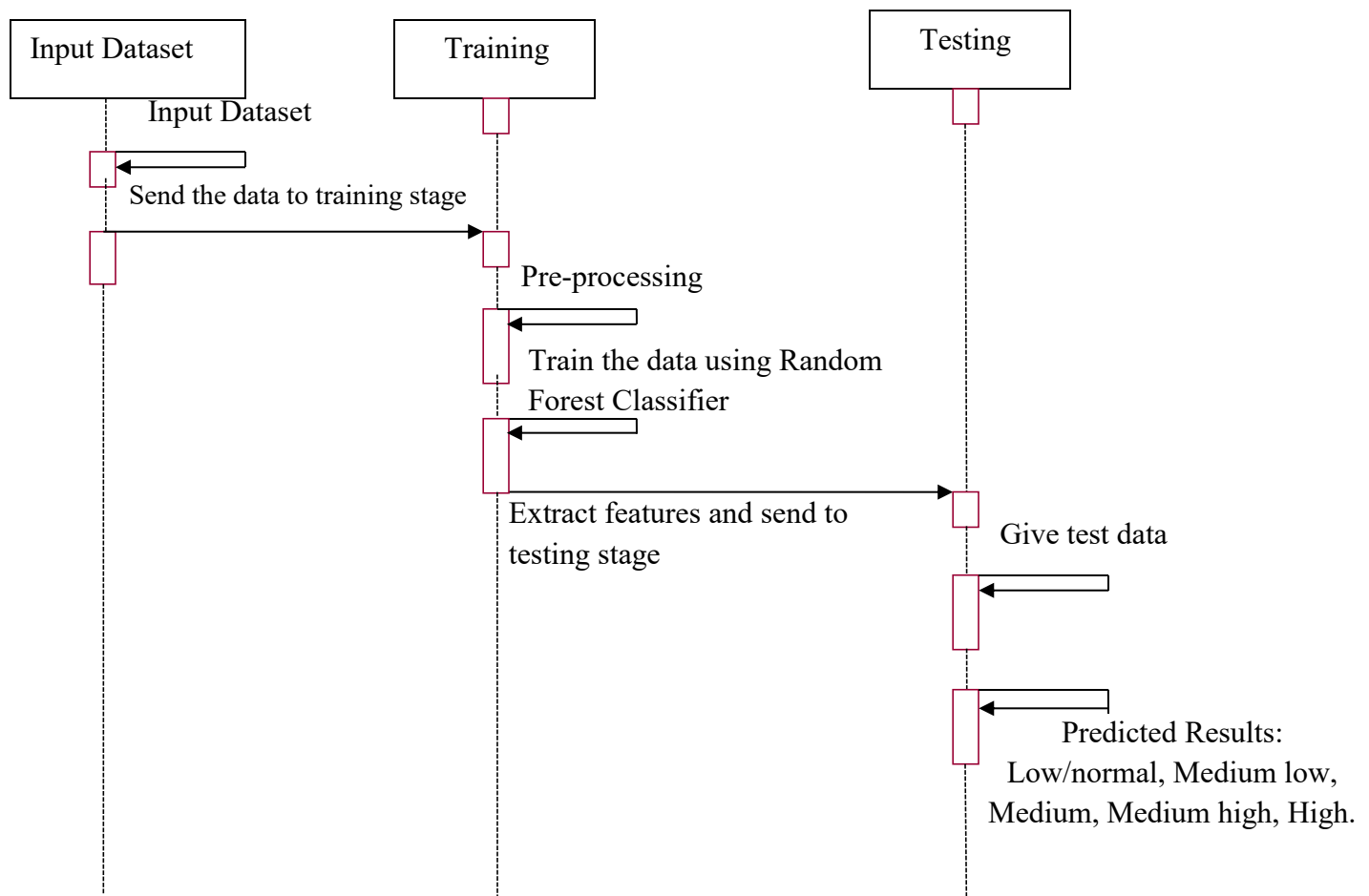
## **CLASS DIAGRAM:**

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



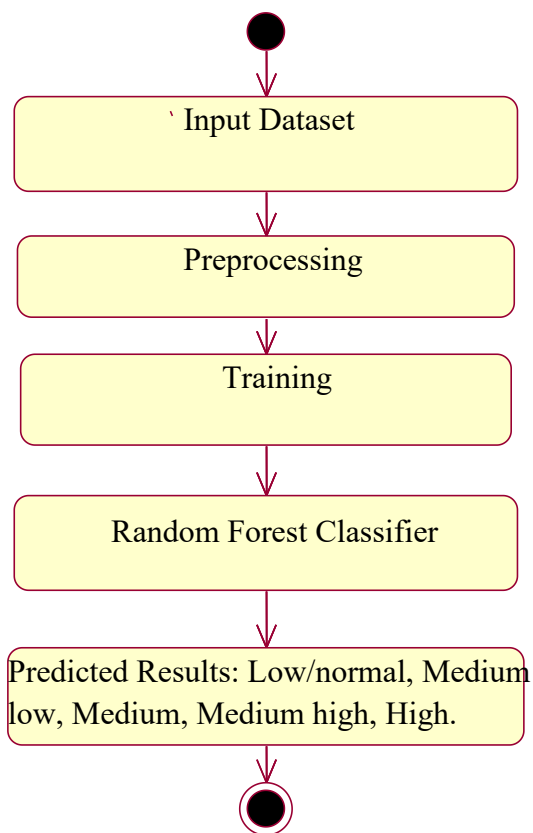
## **SEQUENCE DIAGRAM:**

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



### **ACTIVITY DIAGRAM:**

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



## **SOFTWARE ENVIRONMENT**

### **Python:**

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

### **History of Python**

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

## **Python Features**

Python's features include –

- **Easy-to-learn** – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** – Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- **A broad standard library** – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** – Python provides interfaces to all major commercial databases.

- **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** – Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below –

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

Python is available on a wide variety of platforms including Linux and Mac OS X. Let's understand how to set up our Python environment.

## **Getting Python**

The most up-to-date and current source code, binaries, documentation, news, etc., is available on the official website of Python <https://www.python.org>.

## **Windows Installation**

Here are the steps to install Python on Windows machine.

- Open a Web browser and go to <https://www.python.org/downloads/>.
- Follow the link for the Windows installer python-XYZ.msifile where XYZ is the version you need to install.
- To use this installer python-XYZ.msi, the Windows system must support Microsoft Installer 2.0. Save the installer file to your local machine and then run it to find out if your machine supports MSI.
- Run the downloaded file. This brings up the Python install wizard, which is really easy to use. Just accept the default settings, wait until the install is finished, and you are done.

The Python language has many similarities to Perl, C, and Java. However, there are some definite differences between the languages.

### **First Python Program**

Let us execute programs in different modes of programming.

### **Interactive Mode Programming**

Invoking the interpreter without passing a script file as a parameter brings up the following prompt –

```
$ python
Python2.4.3(#1,Nov112010,13:34:43)
[GCC 4.1.220080704(RedHat4.1.2-48)] on linux2
Type"help","copyright","credits"or"license"for more information.
>>>
```

Type the following text at the Python prompt and press the Enter –

```
>>>print"Hello, Python!"
```

If you are running new version of Python, then you would need to use print statement with parenthesis as in **print ("Hello, Python!");**. However in Python version 2.4.3, this produces the following result –

```
Hello, Python!
```

## Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension **.py**. Type the following source code in a test.py file –

```
print"Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows –

```
$ python test.py
```

This produces the following result –

```
Hello, Python!
```

## Flask Framework:

Flask is a web application framework written in Python. Armin Ronacher, who leads an international group of Python enthusiasts named Pocco, develops it. Flask is based on Werkzeug WSGI toolkit and Jinja2 template engine. Both are Pocco projects.

Http protocol is the foundation of data communication in world wide web. Different methods of data retrieval from specified URL are defined in this protocol.

The following table summarizes different http methods –

| Sr.No | Methods & Description                                                                                           |
|-------|-----------------------------------------------------------------------------------------------------------------|
| 1     | <b>GET</b><br><br>Sends data in unencrypted form to the server. Most common method.                             |
| 2     | <b>HEAD</b><br><br>Same as GET, but without response body                                                       |
| 3     | <b>POST</b><br><br>Used to send HTML form data to server. Data received by POST method is not cached by server. |

|   |                                                                                                                 |
|---|-----------------------------------------------------------------------------------------------------------------|
| 4 | <p><b>PUT</b></p> <p>Replaces all current representations of the target resource with the uploaded content.</p> |
| 5 | <p><b>DELETE</b></p> <p>Removes all current representations of the target resource given by a URL</p>           |

By default, the Flask route responds to the **GET** requests. However, this preference can be altered by providing methods argument to **route()** decorator.

In order to demonstrate the use of **POST** method in URL routing, first let us create an HTML form and use the **POST** method to send form data to a URL.

Save the following script as login.html

```
<html>

<body>

<form action="http://localhost:5000/login" method="post">

<p>Enter Name:</p>

<p><input type="text" name="nm"/></p>

<p><input type="submit" value="submit"/></p>

</form>

</body>
```

</html>

Now enter the following script in Python shell.

```
from flask import Flask, redirect, url_for, request

app = Flask(__name__)

@app.route('/success/<name>')

def success(name):

    return 'welcome %s' % name

@app.route('/login', methods=['POST', 'GET'])

def login():

    if request.method == 'POST':

        user = request.form['nm']

        return redirect(url_for('success', name = user))

    else:

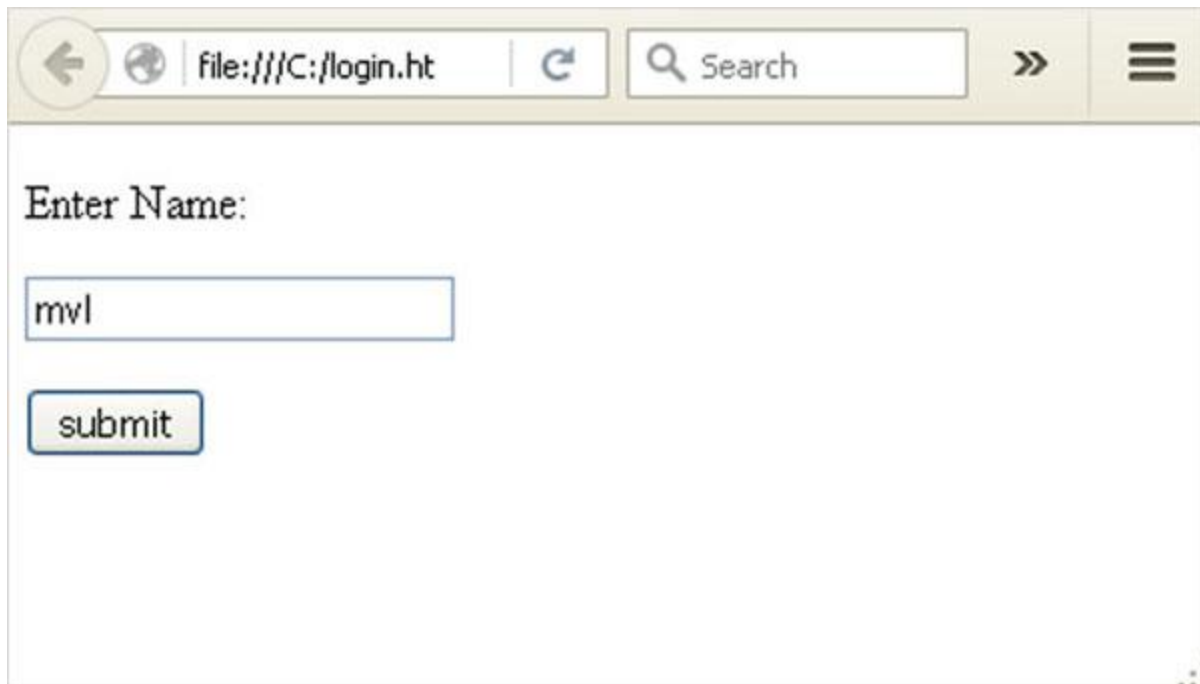
        user = request.args.get('nm')

        return redirect(url_for('success', name = user))

if __name__ == '__main__':

    app.run(debug = True)
```

After the development server starts running, open **login.html** in the browser, enter name in the text field and click **Submit**.

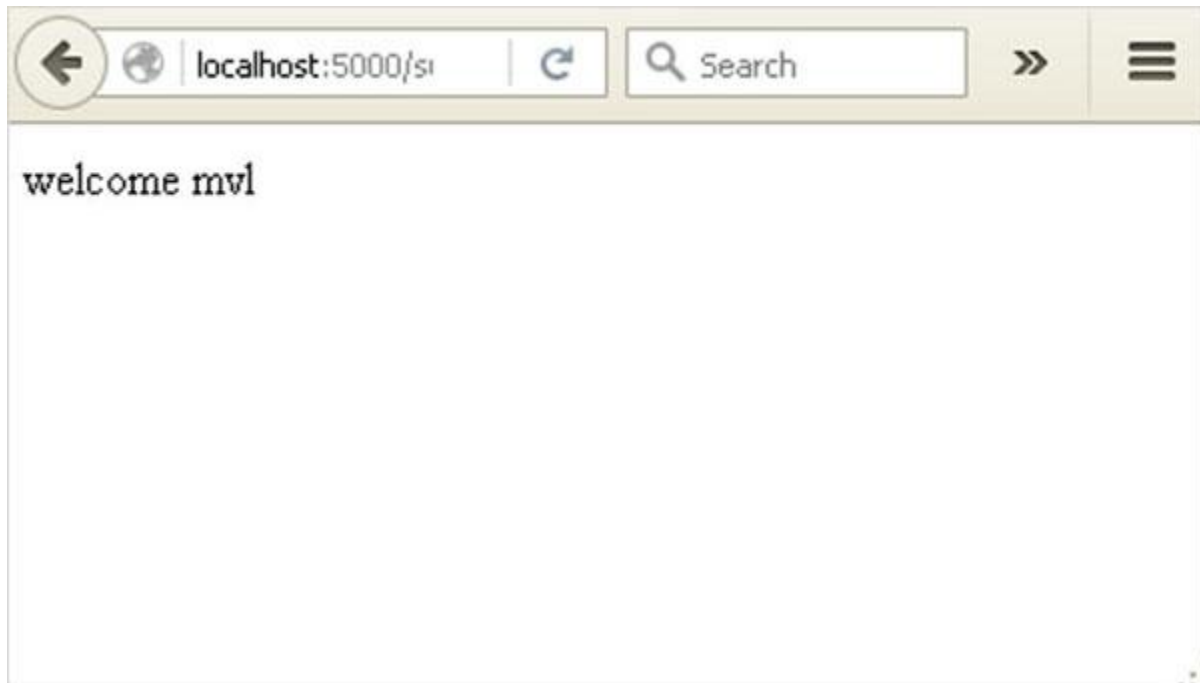
A screenshot of a web browser window. The address bar shows the file path 'file:///C:/login.ht'. The page content includes the text 'Enter Name:', a text input field containing 'mvl', and a 'submit' button.

Form data is POSTed to the URL in action clause of form tag.

**http://localhost/login** is mapped to the **login()** function. Since the server has received data by **POST** method, value of 'nm' parameter obtained from the form data is obtained by –

```
user = request.form['nm']
```

It is passed to **‘/success’** URL as variable part. The browser displays a **welcome** message in the window.



Change the method parameter to **‘GET’** in **login.html** and open it again in the browser. The data received on server is by the **GET** method. The value of **‘nm’** parameter is now obtained by –

```
User = request.args.get('nm')
```

Here, **args** is dictionary object containing a list of pairs of form parameter and its corresponding value. The value corresponding to **‘nm’** parameter is passed on to **‘/success’** URL as before.

## **What is Python?**

Python is a popular programming language. It was created in 1991 by Guido van Rossum.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

## **What can Python do?**

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

## **Why Python?**

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-orientated way or a functional way.

Good to know

- The most recent major version of Python is Python 3, which we shall be using in this tutorial. However, Python 2, although not being updated with anything other than security updates, is still quite popular.
- In this tutorial Python will be written in a text editor. It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

### Python Syntax compared to other programming languages

- Python was designed to for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

## Python Install

Many PCs and Macs will have python already installed.

To check if you have python installed on a Windows PC, search in the start bar for Python or run the following on the Command Line (cmd.exe):

```
C:\Users\Your Name>python --version
```

To check if you have python installed on a Linux or Mac, then on linux open the command line or on Mac open the Terminal and type:

```
python --version
```

If you find that you do not have python installed on your computer, then you can download it for free from the following website: <https://www.python.org/>

# Python Quickstart

Python is an interpreted programming language, this means that as a developer you write Python (.py) files in a text editor and then put those files into the python interpreter to be executed.

The way to run a python file is like this on the command line:

```
C:\Users\Your Name>python helloworld.py
```

Where "helloworld.py" is the name of your python file.

Let's write our first Python file, called helloworld.py, which can be done in any text editor.

```
helloworld.py
```

```
print("Hello, World!")
```

Simple as that. Save your file. Open your command line, navigate to the directory where you saved your file, and run:

```
C:\Users\Your Name>python helloworld.py
```

The output should read:

```
Hello, World!
```

Congratulations, you have written and executed your first Python program.

---

## The Python Command Line

To test a short amount of code in python sometimes it is quickest and easiest not to write the code in a file. This is made possible because Python can be run as a command line itself.

Type the following on the Windows, Mac or Linux command line:

```
C:\Users\Your Name>python
```

From there you can write any python, including our hello world example from earlier in the tutorial:

```
C:\Users\Your Name>python
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
```

Which will write "Hello, World!" in the command line:

```
C:\Users\Your Name>python
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
```

Whenever you are done in the python command line, you can simply type the following to quit the python command line interface:

```
exit()
```

## Execute Python Syntax

As we learned in the previous page, Python syntax can be executed by writing directly in the Command Line:

```
>>>                                     print("Hello,                               World!")
Hello, World!
```

Or by creating a python file on the server, using the .py file extension, and running it in the Command Line:

```
C:\Users\Your Name>python myfile.py
```

---

## Python Indentations

Where in other programming languages the indentation in code is for readability only, in Python the indentation is very important.

Python uses indentation to indicate a block of code.

### Example

```
if 5 > 2:
    print("Five is greater than two!")
```

Python will give you an error if you skip the indentation:

### Example

```
if 5 > 2:
print("Five is greater than two!")
```

---

### Comments

Python has commenting capability for the purpose of in-code documentation.

Comments start with a #, and Python will render the rest of the line as a comment:

### Example

Comments in Python:

```
#This is a comment.
print("Hello, World!")
```

### Docstrings

Python also has extended documentation capability, called docstrings.

Docstrings can be one line, or multiline.

Python uses triple quotes at the beginning and end of the docstring:

### Example

Docstrings are also comments:

```
"""This is a
multiline docstring."""
print("Hello, World!")
```

## **IMPLEMENTATION**

### **MODULES:**

- ❖ Data Collection
- ❖ Dataset
- ❖ Data Preparation
- ❖ Splitting the dataset
- ❖ Model Selection
- ❖ Analyze and Prediction
- ❖ Accuracy on test set
- ❖ Saving the Trained Model

### **MODULES DESCRIPTION:**

#### **Data Collection:**

In the first module of Human Stress Detection Based on Sleeping Habits Using Machine Learning Algorithms, we developed the system to get the input dataset. Data collection process is the first real step towards the real development of a machine learning model, collecting data. This is a critical step that will cascade in how good the model will be, the more and better data that we get; the better our model will perform. There are several techniques to collect the data, like web scraping, manual interventions. Our dataset is placed in the project and it's located in the model folder. The dataset is referred from the popular standard

dataset repository kaggle where all the researchers refer it. The dataset consists of numerical datas. The following is the URL for the dataset referred from kaggle.

Link: <https://www.kaggle.com/datasets/jayaprakashpondy/sayopillow>

### **Dataset:**

The dataset consists of 630 individual data. There are 9 columns in the dataset, which are described below.

**sr** - Snoring Range.

**rr** - Respiration Rate.

**t** - Body Temperature.

**lm** - Limb Movement Rate.

**bo** - Blood Oxygen.

**rem** - Eye Movement.

**sr.1** - Number Of Hours Sleep.

**hr** - Heart Rate.

**sl** - 0- Low/Normal, 1 – Medium Low, 2- Medium, 3-Medium High, 4 –High.

### **Data Preparation:**

Wrangle data and prepare it for training. Clean that which may require it (remove duplicates, correct errors, deal with missing values, normalization, data type conversions, etc.). Randomize data, which erases the effects of the particular order in which we collected and/or otherwise prepared our data. Visualize data to help detect relevant relationships between variables or class imbalances (bias alert!), or perform other exploratory analysis. Split into training and evaluation sets

### **Splitting the dataset:**

In this module, the image dataset will be divided into training and testing sets. Split the dataset into Train and Test. 80% train data and 20% test data. This will be done to train the model on a subset of the data, validate the model's performance, and test the model on unseen data to evaluate its accuracy. Split the dataset into train and test. 80% train data and 20% test data.

### **Model Selection:**

We used Random Forest Classifier machine learning algorithm, we got a accuracy of 97.6% on test set so we implemented this algorithm.

### **The Random Forests Algorithm**

Let's understand the algorithm in layman's terms. Suppose you want to go on a trip and you would like to travel to a place which you will enjoy.

So what do you do to find a place that you will like? You can search online, read reviews on travel blogs and portals, or you can also ask your friends.

Let's suppose you have decided to ask your friends, and talked with them about their past travel experience to various places. You will get some recommendations from every friend. Now you have to make a list of those recommended places. Then, you ask them to vote (or select one best place for the trip) from the list of recommended places you made. The place with the highest number of votes will be your final choice for the trip.

In the above decision process, there are two parts. First, asking your friends about their individual travel experience and getting one recommendation out of multiple places they have visited. This part is like using the decision tree algorithm. Here, each friend makes a selection of the places he or she has visited so far.

The second part, after collecting all the recommendations, is the voting procedure for selecting the best place in the list of recommendations. This whole process of getting recommendations from friends and voting on them to find the best place is known as the random forests algorithm.

It technically is an ensemble method (based on the divide-and-conquer approach) of decision trees generated on a randomly split dataset. This collection of decision tree classifiers is also known as the forest. The individual decision trees are generated using an attribute selection indicator such as information gain, gain

ratio, and Gini index for each attribute. Each tree depends on an independent random sample. In a classification problem, each tree votes and the most popular class is chosen as the final result. In the case of regression, the average of all the tree outputs is considered as the final result. It is simpler and more powerful compared to the other non-linear classification algorithms.

*How does the algorithm work?*

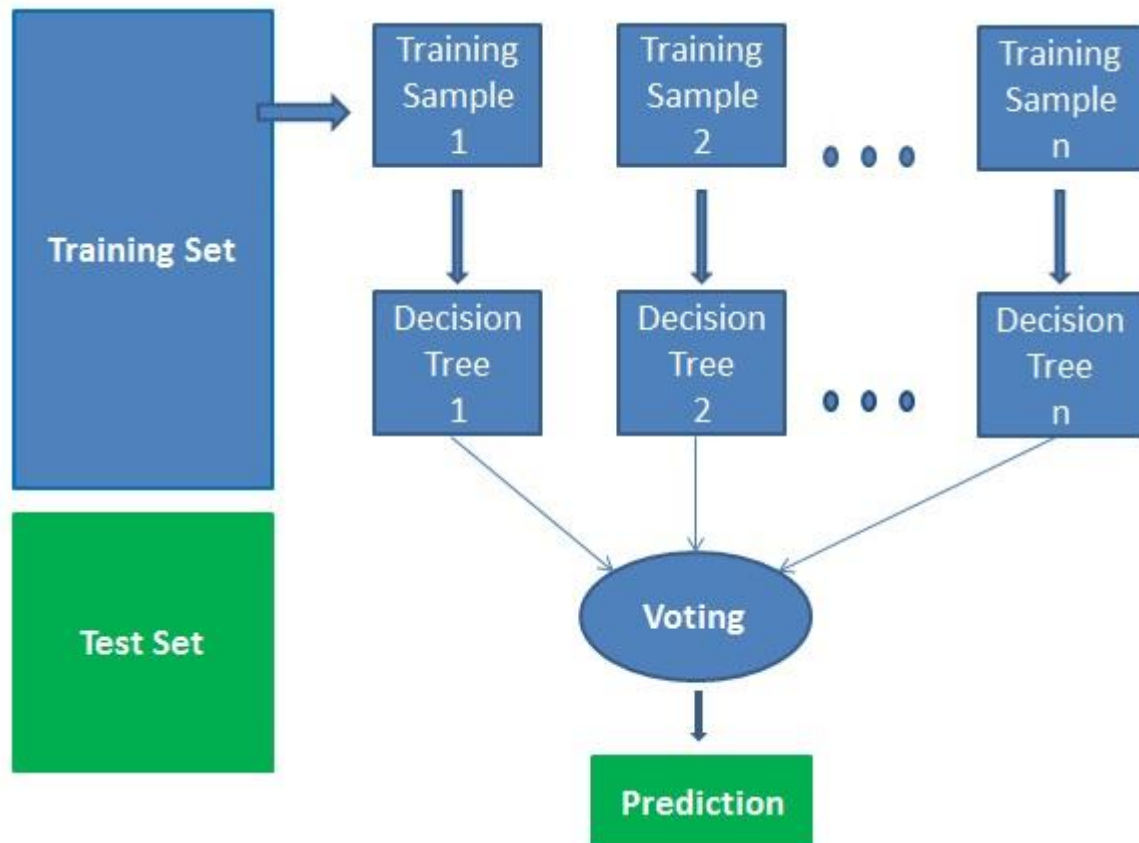
It works in four steps:

Select random samples from a given dataset.

Construct a decision tree for each sample and get a prediction result from each decision tree.

Perform a vote for each predicted result.

Select the prediction result with the most votes as the final prediction.



### *Finding important features*

Random forests also offer a good feature selection indicator. Scikit-learn provide an extra variable with the model, which shows the relative importance or contribution of each feature in the prediction. It automatically computes the relevance score of each feature in the training phase. Then it scales the relevance down so that the sum of all scores is 1.

This score will help you choose the most important features and drop the least important ones for model building.

Random forest uses gini importance or mean decrease in impurity (MDI) to calculate the importance of each feature. Gini importance is also known as the

total decrease in node impurity. This is how much the model fit or accuracy decreases when you drop a variable. The larger the decrease, the more significant the variable is. Here, the mean decrease is a significant parameter for variable selection. The Gini index can describe the overall explanatory power of the variables.

### **Analyze and Prediction:**

In the actual dataset, we chose only 8 features:

**sr** - Snoring Range.

**rr** - Respiration Rate.

**t** - Body Temperature.

**lm** - Limb Movement Rate.

**bo** - Blood Oxygen.

**rem** - Eye Movement.

**sr.1** - Number Of Hours Sleep.

**hr** - Heart Rate.

**sl** - 0- Low/Normal, 1 – Medium Low, 2- Medium, 3-Medium High, 4 –High.

Accuracy on test set:

After training and evaluating the model on the validation set, the accuracy of the model will be assessed on the test set. The accuracy on the test set will be an important metric for evaluating the model's performance. We got an accuracy of 97.6% on test set.

### **Saving the Trained Model:**

Once you're confident enough to take your trained and tested model into the production-ready environment, the first step is to save it into a .h5 or .pkl file using a library like `pickle`.

Make sure you have `pickle` installed in your environment.

Next, let's import the module and dump the model into .pkl file

## SAMPLE CODINGS

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
train = pd.read_csv('SaYoPillow.csv')
train.shape
(630, 9)
train
```

| sr  | rr     | t      | lm     | bo     | rem    | sr.1  | hr    | sl      |
|-----|--------|--------|--------|--------|--------|-------|-------|---------|
| 0   | 93.800 | 25.680 | 91.840 | 16.600 | 89.840 | 99.60 | 1.840 | 74.20 3 |
| 1   | 91.640 | 25.104 | 91.552 | 15.880 | 89.552 | 98.88 | 1.552 | 72.76 3 |
| 2   | 60.000 | 20.000 | 96.000 | 10.000 | 95.000 | 85.00 | 7.000 | 60.00 1 |
| 3   | 85.760 | 23.536 | 90.768 | 13.920 | 88.768 | 96.92 | 0.768 | 68.84 3 |
| 4   | 48.120 | 17.248 | 97.872 | 6.496  | 96.248 | 72.48 | 8.248 | 53.12 0 |
| ... | ...    | ...    | ...    | ...    | ...    | ...   | ...   | ...     |
| 625 | 69.600 | 20.960 | 92.960 | 10.960 | 90.960 | 89.80 | 3.440 | 62.40 2 |
| 626 | 48.440 | 17.376 | 98.064 | 6.752  | 96.376 | 73.76 | 8.376 | 53.44 0 |

|     |        |        |        |        |        |                   |
|-----|--------|--------|--------|--------|--------|-------------------|
| 627 | 97.504 | 27.504 | 86.880 | 17.752 | 84.256 | 101.88            |
|     | 0.000  | 78.76  | 4      |        |        |                   |
| 628 | 58.640 | 19.728 | 95.728 | 9.728  | 94.592 | 84.32 6.728 59.32 |
|     | 1      |        |        |        |        |                   |
| 629 | 73.920 | 21.392 | 93.392 | 11.392 | 91.392 | 91.96 4.088       |
|     | 63.48  | 2      |        |        |        |                   |

630 rows × 9 columns

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 630 entries, 0 to 629
```

```
Data columns (total 9 columns):
```

```
#   Column  Non-Null Count  Dtype
```

```
---  -----  -
```

```
0  sr    630 non-null  float64
1  rr    630 non-null  float64
2  t     630 non-null  float64
3  lm    630 non-null  float64
4  bo    630 non-null  float64
5  rem   630 non-null  float64
6  sr.1  630 non-null  float64
7  hr    630 non-null  float64
8  sl    630 non-null  int64
```

```
dtypes: float64(8), int64(1)
```

```
memory usage: 44.4 KB
```

```
# sr - snoring range, rr - respiration rate, t - body temperature, lm - limb
movement rate, bo - blood oxygen,rem - eye movement,sr.1 - number of hours
```

sleep,hr - heart rate, sl - 0- low/normal, 1 – medium low, 2- medium, 3-medium high, 4 -high

```
train.rename(columns={"sr": "Snoring Range", "rr" : "Respiration Rate", "t" : "Body Temperature", 'lm' : "Limb Movement Rate", "bo" : "Blood Oxygen", "rem" : "Eye Movement", "sr.1" : "Number Of Hours Sleep", "hr": "Heart Rate", "sl": "Stress Level"}, inplace=True)
```

train

|     | Snoring range | respiration rate | Body Temperature | Limb Movement | Rate   | Blood Oxygen | Eye Movement | Number Of Hours Sleep | Heart Rate | Stress Level |
|-----|---------------|------------------|------------------|---------------|--------|--------------|--------------|-----------------------|------------|--------------|
| 0   | 93.800        | 25.680           | 91.840           | 16.600        | 89.840 | 99.60        | 1.840        | 74.20                 | 3          |              |
| 1   | 91.640        | 25.104           | 91.552           | 15.880        | 89.552 | 98.88        | 1.552        | 72.76                 | 3          |              |
| 2   | 60.000        | 20.000           | 96.000           | 10.000        | 95.000 | 85.00        | 7.000        | 60.00                 | 1          |              |
| 3   | 85.760        | 23.536           | 90.768           | 13.920        | 88.768 | 96.92        | 0.768        | 68.84                 | 3          |              |
| 4   | 48.120        | 17.248           | 97.872           | 6.496         | 96.248 | 72.48        | 8.248        | 53.12                 | 0          |              |
| ... | ...           | ...              | ...              | ...           | ...    | ...          | ...          | ...                   | ...        |              |
| 625 | 69.600        | 20.960           | 92.960           | 10.960        | 90.960 | 89.80        | 3.440        | 62.40                 | 2          |              |
| 626 | 48.440        | 17.376           | 98.064           | 6.752         | 96.376 | 73.76        | 8.376        | 53.44                 | 0          |              |
| 627 | 97.504        | 27.504           | 86.880           | 17.752        | 84.256 | 101.88       | 0.000        | 78.76                 | 4          |              |

|         |        |        |        |        |        |       |       |       |
|---------|--------|--------|--------|--------|--------|-------|-------|-------|
| 628     | 58.640 | 19.728 | 95.728 | 9.728  | 94.592 | 84.32 | 6.728 | 59.32 |
| 1       |        |        |        |        |        |       |       |       |
| 629     | 73.920 | 21.392 | 93.392 | 11.392 | 91.392 | 91.96 | 4.088 |       |
| 63.48 2 |        |        |        |        |        |       |       |       |

630 rows × 9 columns

```
train['Snoring range'].unique()
```

```
array([ 93.8 , 91.64 , 60.  , 85.76 , 48.12 , 56.88 , 47.  ,
        50.  , 45.28 , 55.52 , 73.44 , 59.28 , 48.6 , 96.288,
        87.8 , 52.32 , 52.64 , 86.24 , 81.56 , 63.68 , 77.6 ,
        77.28 , 69.76 , 88.04 , 89.96 , 53.68 , 78.56 , 50.96 ,
        94.76 , 62.08 , 49.12 , 96.256, 47.68 , 99.904, 86.72 ,
        48.48 , 98.528, 46.12 , 98.72 , 60.96 , 65.6 , 75.36 ,
        49.28 , 73.28 , 98.912, 62.88 , 97.664, 56.72 , 96.576,
        49.6 , 53.6 , 93.2 , 46.84 , 56.08 , 75.68 , 97.536,
        92.84 , 87.56 , 58.48 , 98.336, 81.08 , 46.6 , 61.6 ,
        96.448, 50.24 , 49.92 , 60.48 , 99.136, 48.28 , 50.48 ,
        96.192, 57.68 , 45.16 , 48.68 , 98.016, 96.8 , 56.48 ,
        90.08 , 96.  , 50.64 , 50.56 , 53.92 , 96.672, 68.32 ,
        54.88 , 52.4 , 51.6 , 92.48 , 98.24 , 65.12 , 86.6 ,
        68.  , 99.808, 45.2 , 63.2 , 98.272, 75.2 , 76.96 ,
        80.  , 99.616, 97.216, 45.04 , 97.376, 53.52 , 61.44 ,
        62.24 , 79.2 , 96.96 , 54.4 , 68.96 , 96.608, 47.44 ,
        90.56 , 66.88 , 97.92 , 68.64 , 76.  , 49.52 , 54.96 ,
        55.28 , 90.32 , 97.312, 58.08 , 83.6 , 98.112, 98.208,
        65.76 , 57.2 , 97.568, 68.8 , 47.8 , 97.184, 92.72 ,
        59.68 , 48.56 , 48.36 , 78.08 , 96.736, 47.72 , 71.36 ,
```

81.44 , 78.4 , 49.32 , 68.48 , 46.92 , 48.76 , 45.08 ,  
45.12 , 50.32 , 88.52 , 47.88 , 96.864, 96.096, 49. ,  
64.64 , 93.32 , 74.56 , 96.32 , 98.784, 98.88 , 70.56 ,  
55.76 , 66.08 , 99.392, 76.8 , 46.8 , 47.28 , 99.776,  
100. , 59.6 , 74.72 , 71.2 , 58.16 , 53.44 , 52.56 ,  
54.32 , 98.08 , 92. , 58.88 , 55.2 , 53.36 , 79.04 ,  
58.24 , 46.2 , 67.04 , 47.04 , 73.76 , 57.04 , 72.48 ,  
52.8 , 57.28 , 96.384, 88.4 , 98.304, 46.68 , 45.96 ,  
48.64 , 67.36 , 88.76 , 46.36 , 45.44 , 48.84 , 96.64 ,  
45.32 , 67.68 , 99.552, 58.32 , 52.08 , 56.56 , 82.28 ,  
45.48 , 48.72 , 93.08 , 87.44 , 47.4 , 77.76 , 65.28 ,  
61.76 , 59.36 , 97.088, 47.2 , 55.68 , 80.12 , 96.768,  
61.12 , 97.408, 45.52 , 91.4 , 97.056, 57.44 , 90.44 ,  
46.76 , 45.84 , 88.64 , 98.048, 83.12 , 56.64 , 98.176,  
93.44 , 84.44 , 98.432, 80.96 , 49.08 , 86.96 , 79.36 ,  
81.2 , 83.96 , 51.36 , 97.248, 60.16 , 82.76 , 51.12 ,  
92.96 , 74.88 , 83.48 , 97.632, 69.44 , 99.872, 81.92 ,  
73.12 , 48.88 , 47.12 , 56.16 , 77.44 , 84.8 , 74.4 ,  
77.92 , 98.848, 99.04 , 58.56 , 46.64 , 98.656, 51.92 ,  
70.08 , 89.48 , 96.928, 89.36 , 94.28 , 83.36 , 45.36 ,  
94.88 , 47.24 , 54.56 , 89.84 , 89. , 88.28 , 64.32 ,  
96.48 , 48.24 , 97.344, 45.88 , 98.368, 49.68 , 69.12 ,  
59.52 , 76.48 , 53.12 , 86.36 , 49.36 , 46. , 91.76 ,  
65.92 , 62.56 , 96.704, 72.32 , 90.92 , 48.8 , 85.04 ,  
71.04 , 55.92 , 45.56 , 56.8 , 49.8 , 95. , 87.2 ,  
82.52 , 48. , 91.16 , 96.032, 71.52 , 96.064, 89.6 ,  
99.68 , 57.6 , 70.72 , 49.4 , 74.08 , 96.992, 49.2 ,

51.04 , 84.68 , 98.464, 46.24 , 50.72 , 47.52 , 47.08 ,  
89.12 , 45.6 , 45.68 , 64.16 , 92.12 , 72.64 , 97.28 ,  
49.56 , 59.44 , 92.36 , 97.44 , 80.72 , 99.936, 99.168,  
85.16 , 97.696, 75.84 , 45. , 98.496, 64. , 81.32 ,  
75.52 , 52. , 66.4 , 71.84 , 99.072, 49.24 , 86.48 ,  
46.04 , 58.96 , 92.24 , 97.472, 93.92 , 67.84 , 46.28 ,  
90.2 , 50.4 , 70.88 , 97.792, 47.96 , 56.24 , 96.16 ,  
55.12 , 61.92 , 48.08 , 84.32 , 47.32 , 53.04 , 57.12 ,  
84.92 , 55.04 , 99.968, 57.92 , 96.896, 50.8 , 98.56 ,  
51.84 , 49.72 , 83.84 , 86.84 , 85.4 , 80.36 , 85.64 ,  
96.512, 78.72 , 62.72 , 84.2 , 56.4 , 83.24 , 90.68 ,  
76.16 , 48.2 , 97.856, 98.688, 52.24 , 50.16 , 45.72 ,  
59.12 , 97.152, 81.8 , 49.88 , 52.16 , 98.624, 98.4 ,  
47.84 , 48.52 , 56. , 58.72 , 48.04 , 99.744, 47.36 ,  
97.824, 66.56 , 49.84 , 72.16 , 84.08 , 72.8 , 77.12 ,  
99.456, 88.16 , 52.88 , 97.888, 54.08 , 57.36 , 51.2 ,  
63.36 , 97.984, 98.752, 82.4 , 66.24 , 49.76 , 59.84 ,  
87.92 , 52.48 , 87.32 , 55.44 , 52.96 , 54.24 , 53.76 ,  
99.584, 78.88 , 97.952, 57.52 , 99.712, 60.8 , 69.92 ,  
51.52 , 91.28 , 86.12 , 76.32 , 56.32 , 45.4 , 48.92 ,  
94.16 , 51.76 , 45.92 , 91.04 , 54.8 , 49.96 , 51.28 ,  
99.36 , 47.76 , 46.96 , 92.6 , 68.16 , 61.28 , 96.832,  
52.72 , 82.16 , 63.04 , 99.232, 63.84 , 99.424, 60.64 ,  
59.2 , 89.72 , 46.56 , 50.08 , 49.16 , 46.88 , 83.72 ,  
63.52 , 96.128, 80.48 , 45.8 , 47.16 , 99.264, 60.32 ,  
96.224, 99.008, 50.88 , 78.24 , 89.24 , 47.92 , 51.68 ,  
46.48 , 82.04 , 49.64 , 46.72 , 67.52 , 53.28 , 45.76 ,

46.4 , 99.52 , 46.44 , 99.2 , 53.84 , 79.52 , 96.544,  
80.6 , 79.68 , 98.944, 47.6 , 81.68 , 59.92 , 48.96 ,  
48.16 , 58.4 , 97.728, 70.24 , 85.28 , 46.32 , 54.48 ,  
87.68 , 99.296, 80.24 , 98.816, 64.48 , 99.648, 64.96 ,  
82.64 , 46.52 , 91.88 , 64.8 , 49.04 , 75.04 , 87.08 ,  
76.64 , 85.88 , 99.488, 51.44 , 58. , 49.48 , 70.4 ,  
72.96 , 94.4 , 54.72 , 80.84 , 94.64 , 91.52 , 97.6 ,  
99.84 , 85.52 , 54.16 , 55.36 , 46.16 , 96.416, 86. ,  
79.84 , 90.8 , 96.352, 55.6 , 72. , 84.56 , 59.76 ,  
83. , 74.24 , 45.24 , 49.44 , 66.72 , 54. , 94.52 ,  
47.56 , 47.64 , 47.48 , 97.12 , 69.28 , 98.976, 45.64 ,  
99.104, 99.328, 93.56 , 57.76 , 56.96 , 55.84 , 48.32 ,  
94.04 , 97.76 , 67.2 , 57.84 , 54.64 , 93.68 , 98.592,  
88.88 , 73.6 , 62.4 , 82.88 , 59.04 , 71.68 , 48.4 ,  
46.08 , 97.024, 53.2 , 65.44 , 98.144, 58.8 , 69.6 ,  
48.44 , 97.504, 58.64 , 73.92 ])

```
train['Heart Rate'].unique()
```

```
array([74.2 , 72.76, 60. , 68.84, 53.12, 58.44, 52. , 55. , 50.28,  
57.76, 63.36, 59.64, 53.6 , 75.72, 70.2 , 56.16, 56.32, 69.16,  
66.04, 60.92, 64.4 , 64.32, 62.44, 70.36, 71.64, 56.84, 64.64,  
55.48, 74.84, 60.52, 54.12, 75.64, 52.68, 84.76, 69.48, 53.48,  
81.32, 51.12, 81.8 , 60.24, 61.4 , 63.84, 54.28, 63.32, 82.28,  
60.72, 79.16, 58.36, 76.44, 54.6 , 56.8 , 73.8 , 51.84, 58.04,  
63.92, 78.84, 73.56, 70.04, 59.24, 80.84, 65.72, 51.6 , 60.4 ,  
76.12, 55.12, 54.92, 60.12, 82.84, 53.28, 55.24, 75.48, 58.84,  
50.16, 53.68, 80.04, 77. , 58.24, 71.72, 75. , 55.32, 55.28,  
56.96, 76.68, 62.08, 57.44, 56.2 , 55.8 , 73.32, 80.6 , 61.28,
```

69.4 , 62. , 84.52, 50.2 , 60.8 , 80.68, 63.8 , 64.24, 65. ,  
84.04, 78.04, 50.04, 78.44, 56.76, 60.36, 60.56, 64.8 , 77.4 ,  
57.2 , 62.24, 76.52, 52.44, 72.04, 61.72, 79.8 , 62.16, 64. ,  
54.52, 57.48, 57.64, 71.88, 78.28, 59.04, 67.4 , 80.28, 80.52,  
61.44, 58.6 , 78.92, 62.2 , 52.8 , 77.96, 73.48, 59.84, 53.56,  
53.36, 64.52, 76.84, 52.72, 62.84, 65.96, 64.6 , 54.32, 62.12,  
51.92, 53.76, 50.08, 50.12, 55.16, 70.68, 52.88, 77.16, 75.24,  
54. , 61.16, 73.88, 63.64, 75.8 , 81.96, 82.2 , 62.64, 57.88,  
61.52, 83.48, 64.2 , 51.8 , 52.28, 84.44, 85. , 59.8 , 63.68,  
62.8 , 59.08, 56.72, 56.28, 57.16, 80.2 , 73. , 59.44, 57.6 ,  
56.68, 64.76, 59.12, 51.2 , 61.76, 52.04, 63.44, 58.52, 63.12,  
56.4 , 58.64, 75.96, 70.6 , 80.76, 51.68, 50.96, 53.64, 61.84,  
70.84, 51.36, 50.44, 53.84, 76.6 , 50.32, 61.92, 83.88, 59.16,  
56.04, 58.28, 66.52, 50.48, 53.72, 73.72, 69.96, 52.4 , 64.44,  
61.32, 60.44, 59.68, 77.72, 52.2 , 57.84, 65.08, 76.92, 60.28,  
78.52, 50.52, 72.6 , 77.64, 58.72, 71.96, 51.76, 50.84, 70.76,  
80.12, 67.08, 58.32, 80.44, 73.96, 67.96, 81.08, 65.64, 54.08,  
69.64, 64.84, 65.8 , 67.64, 55.68, 78.12, 60.04, 66.84, 55.56,  
73.64, 63.72, 67.32, 79.08, 62.36, 84.68, 66.28, 63.28, 53.88,  
52.12, 58.08, 64.36, 68.2 , 63.6 , 64.48, 82.12, 82.6 , 59.28,  
51.64, 81.64, 55.96, 62.52, 71.32, 77.32, 71.24, 74.52, 67.24,  
50.36, 74.92, 52.24, 57.28, 71.56, 71. , 70.52, 61.08, 76.2 ,  
53.24, 78.36, 50.88, 80.92, 54.68, 62.28, 59.76, 64.12, 56.56,  
69.24, 54.36, 51. , 72.84, 61.48, 60.64, 76.76, 63.08, 72.28,  
53.8 , 68.36, 62.76, 57.96, 50.56, 58.4 , 54.8 , 69.8 , 66.68,  
53. , 72.44, 75.08, 62.88, 75.16, 71.4 , 84.2 , 58.8 , 62.68,  
54.4 , 63.52, 77.48, 54.2 , 55.52, 68.12, 81.16, 51.24, 55.36,

52.52, 52.08, 71.08, 50.6 , 50.68, 61.04, 73.08, 63.16, 78.2 ,  
54.56, 59.72, 73.24, 78.6 , 65.48, 84.84, 82.92, 68.44, 79.24,  
63.96, 50. , 81.24, 61. , 65.88, 63.88, 56. , 61.6 , 62.96,  
82.68, 54.24, 69.32, 51.04, 59.48, 73.16, 78.68, 74.28, 61.96,  
51.28, 71.8 , 55.2 , 62.72, 79.48, 52.96, 58.12, 75.4 , 57.56,  
60.48, 53.08, 67.88, 52.32, 56.52, 58.56, 68.28, 57.52, 84.92,  
58.96, 77.24, 55.4 , 81.4 , 55.92, 54.72, 67.56, 69.56, 68.6 ,  
65.24, 68.76, 76.28, 64.68, 60.68, 67.8 , 58.2 , 67.16, 72.12,  
64.04, 53.2 , 79.64, 81.72, 56.12, 55.08, 50.72, 59.56, 77.88,  
66.2 , 54.88, 56.08, 81.56, 81. , 52.84, 53.52, 58. , 59.36,  
53.04, 84.36, 52.36, 79.56, 61.64, 54.84, 63.04, 67.72, 63.2 ,  
64.28, 83.64, 70.44, 56.44, 79.72, 57.04, 58.68, 55.6 , 60.84,  
79.96, 81.88, 66.6 , 61.56, 54.76, 59.92, 70.28, 56.24, 69.88,  
57.72, 56.48, 57.12, 56.88, 83.96, 64.72, 79.88, 58.76, 84.28,  
60.2 , 62.48, 55.76, 72.52, 69.08, 64.08, 58.16, 50.4 , 53.92,  
74.44, 55.88, 50.92, 72.36, 57.4 , 54.96, 55.64, 83.4 , 52.76,  
51.96, 73.4 , 62.04, 60.32, 77.08, 56.36, 66.44, 60.76, 83.08,  
60.96, 83.56, 60.16, 59.6 , 71.48, 51.56, 55.04, 54.16, 51.88,  
67.48, 60.88, 75.32, 65.32, 50.8 , 52.16, 83.16, 60.08, 75.56,  
82.52, 55.44, 64.56, 71.16, 52.92, 55.84, 51.48, 66.36, 54.64,  
51.72, 61.88, 56.64, 50.76, 51.4 , 83.8 , 51.44, 83. , 56.92,  
64.88, 76.36, 65.4 , 64.92, 82.36, 52.6 , 66.12, 59.96, 53.96,  
53.16, 59.2 , 79.32, 62.56, 68.52, 51.32, 57.24, 70.12, 83.24,  
65.16, 82.04, 61.12, 84.12, 61.24, 66.76, 51.52, 72.92, 61.2 ,  
54.04, 63.76, 69.72, 64.16, 68.92, 83.72, 55.72, 59. , 54.48,  
62.6 , 63.24, 74.6 , 57.36, 65.56, 74.76, 72.68, 79. , 84.6 ,  
68.68, 57.08, 57.68, 51.16, 76.04, 69. , 64.96, 72.2 , 75.88,

57.8 , 63. , 68.04, 59.88, 67. , 63.56, 50.24, 54.44, 61.68,  
57. , 74.68, 52.56, 52.64, 52.48, 77.8 , 62.32, 82.44, 50.64,  
82.76, 83.32, 74.04, 58.88, 58.48, 57.92, 53.32, 74.36, 79.4 ,  
61.8 , 58.92, 57.32, 74.12, 81.48, 70.92, 63.4 , 60.6 , 66.92,  
59.52, 62.92, 53.4 , 51.08, 77.56, 56.6 , 61.36, 80.36, 59.4 ,  
62.4 , 53.44, 78.76, 59.32, 63.48])

train['Blood Oxygen'].unique()

array([89.84 , 89.552, 95. , 88.768, 96.248, 94.064, 95.8 , 97. ,  
95.112, 93.656, 91.344, 94.784, 96.44 , 82.432, 89.04 , 92.696,  
92.792, 88.832, 88.208, 90.368, 91.76 , 91.728, 90.976, 89.072,  
89.328, 93.104, 91.856, 92.288, 89.968, 90.208, 96.648, 82.384,  
96.072, 87.856, 90. , 88.896, 96.392, 85.792, 95.448, 86.08 ,  
90.096, 90.56 , 91.536, 96.712, 91.328, 86.368, 90.288, 84.496,  
94.016, 82.864, 96.84 , 93.08 , 89.76 , 95.736, 93.824, 91.568,  
84.304, 89.712, 89.008, 94.544, 85.504, 88.144, 95.64 , 90.16 ,  
82.672, 92.072, 96.968, 90.048, 86.704, 96.312, 92.144, 82.288,  
94.304, 95.064, 96.472, 85.024, 83.2 , 93.944, 89.344, 82. ,  
92.192, 92.168, 93.176, 83.008, 90.832, 93.464, 92.72 , 92.48 ,  
89.664, 85.36 , 90.512, 88.88 , 90.8 , 87.712, 95.08 , 90.32 ,  
85.408, 91.52 , 91.696, 88. , 87.424, 83.824, 95.016, 84.064,  
93.056, 90.144, 90.224, 91.92 , 83.44 , 93.32 , 90.896, 82.912,  
95.976, 89.408, 90.688, 84.88 , 90.864, 91.6 , 96.808, 93.488,  
93.584, 89.376, 83.968, 94.424, 88.48 , 85.168, 85.312, 90.576,  
94.16 , 84.352, 90.88 , 96.12 , 83.776, 89.696, 94.904, 96.424,  
96.344, 91.808, 83.104, 96.088, 91.136, 88.192, 91.84 , 96.728,  
90.848, 95.768, 96.504, 95.032, 95.048, 92.096, 89.136, 96.152,  
83.296, 82.144, 96.6 , 90.464, 89.776, 91.456, 82.48 , 86.176,

86.32 , 91.056, 93.728, 90.608, 87.088, 91.68 , 95.72 , 95.912,  
87.664, 94.88 , 91.472, 91.12 , 94.448, 93.032, 92.768, 93.296,  
85.12 , 89.6 , 94.664, 93.56 , 93.008, 91.904, 94.472, 95.48 ,  
90.704, 95.816, 91.376, 94.112, 91.248, 92.84 , 94.184, 82.576,  
89.12 , 85.456, 95.672, 95.384, 96.456, 90.736, 89.168, 95.544,  
95.176, 96.536, 92. , 82.96 , 95.128, 90.768, 87.328, 94.496,  
92.624, 93.968, 88.304, 95.192, 96.488, 89.744, 88.992, 95.96 ,  
91.776, 90.528, 90.176, 94.808, 83.632, 95.88 , 93.704, 88.016,  
83.152, 90.112, 84.112, 95.208, 89.52 , 83.584, 94.232, 89.392,  
95.704, 95.336, 89.152, 85.072, 88.416, 93.992, 85.264, 89.792,  
88.592, 85.648, 88.128, 96.632, 88.928, 91.936, 88.16 , 88.528,  
92.408, 83.872, 90.016, 88.368, 92.336, 89.728, 91.488, 88.464,  
84.448, 90.944, 87.808, 88.256, 91.312, 96.552, 95.848, 93.848,  
91.744, 88.64 , 91.44 , 91.792, 86.272, 86.56 , 94.568, 95.656,  
85.984, 92.576, 91.008, 89.264, 83.392, 89.248, 89.904, 88.448,  
95.144, 89.984, 95.896, 93.368, 89.312, 89.2 , 89.104, 90.432,  
82.72 , 96.296, 84.016, 95.352, 85.552, 96.872, 90.912, 94.856,  
91.648, 92.936, 88.848, 96.744, 95.4 , 89.568, 90.592, 90.256,  
83.056, 91.232, 89.456, 96.52 , 88.672, 91.104, 93.776, 95.224,  
94.04 , 96.92 , 88.96 , 88.336, 96.2 , 89.488, 82.048, 91.152,  
82.096, 89.28 , 87.52 , 94.28 , 91.072, 96.76 , 91.408, 83.488,  
96.68 , 92.312, 88.624, 85.696, 95.496, 92.216, 96.008, 95.832,  
89.216, 95.24 , 95.272, 90.416, 89.616, 91.264, 83.92 , 96.824,  
94.832, 89.648, 84.16 , 88.096, 87.904, 86.752, 88.688, 84.544,  
91.584, 85.744, 90.4 , 88.176, 91.552, 92.6 , 90.64 , 91.184,  
86.608, 96.696, 88.864, 95.416, 94.688, 89.632, 84.208, 89.856,  
90.784, 95.512, 89.36 , 92.12 , 91.088, 84.688, 96.184, 93.872,

82.24 , 93.536, 90.192, 96.232, 88.576, 95.928, 92.912, 94.136,  
88.656, 93.512, 87.952, 94.376, 83.344, 92.24 , 85.84 , 92.552,  
96.888, 88.512, 88.912, 88.72 , 88.048, 88.752, 82.768, 91.872,  
90.272, 88.56 , 93.92 , 88.432, 89.424, 91.616, 96.28 , 84.784,  
86.032, 92.672, 92.048, 95.288, 94.736, 83.728, 88.24 , 96.952,  
92.648, 85.936, 85.6 , 96.136, 96.408, 93.8 , 94.616, 96.216,  
87.616, 95.944, 84.736, 90.656, 96.936, 91.216, 88.544, 91.28 ,  
91.712, 87.184, 89.088, 92.864, 84.832, 93.224, 94.208, 92.36 ,  
90.336, 84.976, 86.128, 88.32 , 90.624, 96.904, 94.952, 89.056,  
92.744, 88.976, 93.632, 92.888, 93.272, 93.128, 87.376, 91.888,  
84.928, 94.256, 87.568, 90.08 , 90.992, 92.456, 89.504, 88.816,  
91.632, 93.896, 95.16 , 96.568, 89.888, 92.528, 95.368, 89.472,  
93.44 , 96.984, 92.384, 87.04 , 96.104, 95.784, 89.68 , 90.816,  
90.128, 83.248, 92.816, 88.288, 90.304, 86.848, 90.384, 87.136,  
90.064, 94.76 , 89.296, 95.624, 92.024, 96.664, 95.752, 88.496,  
90.352, 82.192, 88.064, 95.32 , 95.864, 86.896, 90.032, 82.336,  
86.512, 92.264, 91.824, 89.232, 96.168, 92.504, 95.592, 88.272,  
96.856, 95.688, 90.752, 92.984, 95.304, 95.56 , 87.28 , 95.576,  
86.8 , 93.152, 91.952, 82.816, 88.08 , 91.968, 86.416, 96.04 ,  
88.224, 94.976, 96.584, 96.264, 94.52 , 84.592, 91.024, 88.704,  
95.528, 93.344, 89.024, 86.944, 88.032, 86.224, 90.448, 87.472,  
90.496, 88.352, 95.608, 89.584, 90.48 , 96.616, 91.504, 88.944,  
91.664, 88.784, 87.232, 92.432, 94.4 , 96.792, 91.04 , 91.296,  
89.92 , 93.416, 88.112, 89.952, 89.536, 84.4 , 87.76 , 88.736,  
93.248, 93.608, 95.464, 82.624, 88.8 , 91.984, 89.44 , 82.528,  
93.68 , 91.2 , 88.608, 94.928, 88.4 , 91.424, 95.096, 96.776,  
90.672, 93.2 , 89.936, 96.024, 96.056, 95.992, 83.68 , 90.928,

```
86.464, 95.256, 86.656, 86.992, 89.808, 94.328, 94.088, 93.752,  
96.328, 89.872, 84.64 , 90.72 , 94.352, 93.392, 89.824, 85.888,  
89.184, 91.36 , 90.24 , 88.384, 94.712, 91.168, 96.36 , 95.432,  
83.536, 92.96 , 90.544, 85.216, 94.64 , 90.96 , 96.376, 84.256,  
94.592, 91.392])
```

```
train['Stress Level'].unique()
```

```
array([3, 1, 0, 2, 4], dtype=int64)
```

```
train['Stress Level'].value_counts()
```

```
3    126
```

```
1    126
```

```
0    126
```

```
2    126
```

```
4    126
```

```
Name: Stress Level, dtype: int64
```

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 630 entries, 0 to 629
```

```
Data columns (total 9 columns):
```

| # | Column                | Non-Null Count | Dtype   |
|---|-----------------------|----------------|---------|
| 0 | Snoring range         | 630 non-null   | float64 |
| 1 | respiration rate      | 630 non-null   | float64 |
| 2 | Body Temperature      | 630 non-null   | float64 |
| 3 | Limb Movment Rate     | 630 non-null   | float64 |
| 4 | Blood Oxygen          | 630 non-null   | float64 |
| 5 | Eye Movement          | 630 non-null   | float64 |
| 6 | Number Of Hours Sleep | 630 non-null   | float64 |

```
7 Heart Rate      630 non-null  float64
8 Stress Level    630 non-null  int64
```

```
dtypes: float64(8), int64(1)
```

```
memory usage: 44.4 KB
```

```
UNIVARIATE ANALYSIS
```

```
NUMERICAL DATA ANALYSIS
```

```
sns.distplot(train["Body Temperature"])
```

```
train['Body Temperature'].skew()
```

```
C:\Users\IT      Support\AppData\Local\Temp\ipykernel_25736\1378587863.py:1:
```

```
UserWarning:
```

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see

<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Body Temperature"])
```

```
-0.2857511600166689
```

```
sns.distplot(train["Limb Movment Rate"])
```

```
train['Limb Movment Rate'].skew()
```

C:\Users\IT Support\AppData\Local\Temp\ipykernel\_25736\2987789043.py:1:  
UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see  
<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Limb Movment Rate"])  
0.16266707843026826
```

```
sns.distplot(train["Blood Oxgyen"])  
train['Blood Oxgyen'].skew()
```

C:\Users\IT Support\AppData\Local\Temp\ipykernel\_25736\1651384083.py:1:  
UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see  
<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Blood Oxgyen"])  
-0.3606553282300263
```

```
sns.distplot(train["Eye Movement"])  
train['Eye Movement'].skew()
```

```
C:\Users\IT      Support\AppData\Local\Temp\ipykernel_25736\1126066828.py:1:  
UserWarning:
```

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see  
<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Eye Movement"])  
-0.5738786598112939
```

```
sns.distplot(train["Number Of Hours Sleep"])  
train['Number Of Hours Sleep'].skew()
```

```
C:\Users\IT      Support\AppData\Local\Temp\ipykernel_25736\3804887666.py:1:  
UserWarning:
```

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with

similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see

<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Number Of Hours Sleep"])  
0.17511258649719963
```

```
sns.distplot(train["Heart Rate"])  
train['Heart Rate'].skew()
```

```
C:\Users\IT      Support\AppData\Local\Temp\ipykernel_25736\1959881697.py:1:  
UserWarning:
```

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see

<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Heart Rate"])  
0.4558625471984541
```

```
sns.distplot(train["Snoring range"])  
train['Snoring range'].skew()
```

C:\Users\IT Support\AppData\Local\Temp\ipykernel\_25736\467788349.py:1:  
UserWarning:

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see  
<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train["Snoring range"])  
0.11393461423002128
```

```
sns.distplot(train['respiration rate'])  
train['respiration rate'].skew()
```

C:\Users\IT Support\AppData\Local\Temp\ipykernel\_25736\2753615147.py:1:  
UserWarning:

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see  
<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train['respiration rate'])  
0.45586254719845326
```

```
train[train['Stress Level']==1].shape  
(126, 9)
```

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 630 entries, 0 to 629
```

```
Data columns (total 9 columns):
```

| # | Column                | Non-Null Count | Dtype   |
|---|-----------------------|----------------|---------|
| 0 | Snoring range         | 630 non-null   | float64 |
| 1 | respiration rate      | 630 non-null   | float64 |
| 2 | Body Temperature      | 630 non-null   | float64 |
| 3 | Limb Movment Rate     | 630 non-null   | float64 |
| 4 | Blood Oxygen          | 630 non-null   | float64 |
| 5 | Eye Movement          | 630 non-null   | float64 |
| 6 | Number Of Hours Sleep | 630 non-null   | float64 |
| 7 | Heart Rate            | 630 non-null   | float64 |
| 8 | Stress Level          | 630 non-null   | int64   |

```
dtypes: float64(8), int64(1)
```

```
memory usage: 44.4 KB
```

```
train.head()
```

|               |                  |                       |              |      |
|---------------|------------------|-----------------------|--------------|------|
| Snoring range | respiration rate | Body Temperature      | Limb Movment | Rate |
| Blood Oxygen  | Eye Movement     | Number Of Hours Sleep | Heart        | Rate |
| Stress Level  |                  |                       |              |      |

|   |       |        |        |        |        |       |       |       |
|---|-------|--------|--------|--------|--------|-------|-------|-------|
| 0 | 93.80 | 25.680 | 91.840 | 16.600 | 89.840 | 99.60 | 1.840 | 74.20 |
| 3 |       |        |        |        |        |       |       |       |
| 1 | 91.64 | 25.104 | 91.552 | 15.880 | 89.552 | 98.88 | 1.552 | 72.76 |
| 3 |       |        |        |        |        |       |       |       |
| 2 | 60.00 | 20.000 | 96.000 | 10.000 | 95.000 | 85.00 | 7.000 | 60.00 |
| 1 |       |        |        |        |        |       |       |       |
| 3 | 85.76 | 23.536 | 90.768 | 13.920 | 88.768 | 96.92 | 0.768 | 68.84 |
| 3 |       |        |        |        |        |       |       |       |
| 4 | 48.12 | 17.248 | 97.872 | 6.496  | 96.248 | 72.48 | 8.248 | 53.12 |
| 0 |       |        |        |        |        |       |       |       |

```
sns.boxplot(x='Stress Level', y='Blood Oxgyen', data=train)
```

```
<Axes: xlabel='Stress Level', ylabel='Blood Oxgyen'>
```

```
sns.barplot(x='Stress Level',y='Heart Rate',data=train)
```

```
<Axes: xlabel='Stress Level', ylabel='Heart Rate'>
```

```
train.head()
```

|   | Snoring range | respiration rate | Body Temperature      | Limb Movment | Rate   |
|---|---------------|------------------|-----------------------|--------------|--------|
|   | Blood Oxgyen  | Eye Movement     | Number Of Hours Sleep | Heart        | Rate   |
|   | Stress Level  |                  |                       |              |        |
| 0 | 93.80         | 25.680           | 91.840                | 16.600       | 89.840 |
| 3 |               |                  |                       |              |        |
| 1 | 91.64         | 25.104           | 91.552                | 15.880       | 89.552 |
| 3 |               |                  |                       |              |        |
| 2 | 60.00         | 20.000           | 96.000                | 10.000       | 95.000 |
| 1 |               |                  |                       |              |        |
| 3 | 85.76         | 23.536           | 90.768                | 13.920       | 88.768 |
| 3 |               |                  |                       |              |        |

```
4    48.12 17.248    97.872    6.496 96.248    72.48 8.248 53.12 0
```

```
train.head()
```

```
Snoring range    respiration rate    Body Temperature Limb    Movment    Rate
      Blood Oxygen    Eye Movement    Number Of Hours Sleep    Heart    Rate
Stress Level
```

```
0    93.80 25.680    91.840    16.600    89.840    99.60 1.840 74.20
3
```

```
1    91.64 25.104    91.552    15.880    89.552    98.88 1.552 72.76
3
```

```
2    60.00 20.000    96.000    10.000    95.000    85.00 7.000 60.00
1
```

```
3    85.76 23.536    90.768    13.920    88.768    96.92 0.768 68.84
3
```

```
4    48.12 17.248    97.872    6.496 96.248    72.48 8.248 53.12 0
```

```
X = train.drop(['Stress Level'], axis=1).values
```

```
y = train['Stress Level'].values
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=9)
```

```
print(X_train.shape)
```

```
print(X_test.shape)
```

```
(441, 8)
```

```
(189, 8)
```

```
X
```

```
array([[ 93.8 , 25.68 , 91.84 , ..., 99.6 , 1.84 , 74.2 ],
      [ 91.64 , 25.104, 91.552, ..., 98.88 , 1.552, 72.76 ],
```

```

[ 60. , 20. , 96. , ..., 85. , 7. , 60. ],
...,
[ 97.504, 27.504, 86.88 , ..., 101.88 , 0. , 78.76 ],
[ 58.64 , 19.728, 95.728, ..., 84.32 , 6.728, 59.32 ],
[ 73.92 , 21.392, 93.392, ..., 91.96 , 4.088, 63.48 ]])
X_train
array([[ 88.28 , 24.208, 91.104, ..., 97.76 , 1.104, 70.52 ],
       [ 45.72 , 16.288, 96.432, ..., 62.88 , 7.288, 50.72 ],
       [ 88.04 , 24.144, 91.072, ..., 97.68 , 1.072, 70.36 ],
       ...,
       [ 96.128, 26.128, 85.16 , ..., 100.16 , 0. , 75.32 ],
       [ 92.36 , 25.296, 91.648, ..., 99.12 , 1.648, 73.24 ],
       [ 61.92 , 20.192, 92.192, ..., 85.96 , 2.288, 60.48 ]])
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
rfc = RandomForestClassifier(n_estimators=60, random_state=23)
rfc.fit(X_train,y_train)

RandomForestClassifier
RandomForestClassifier(n_estimators=60, random_state=23)
rfc.score(X_train,y_train)
1.0
from sklearn.metrics import accuracy_score
y_pred = rfc.predict(X_test )
accuracy_score(y_pred,y_test)

```

0.9788359788359788

```
import sklearn.metrics
```

```
print(sklearn.metrics.classification_report(y_test, y_pred))
```

|  | precision | recall | f1-score | support |
|--|-----------|--------|----------|---------|
|--|-----------|--------|----------|---------|

|   |      |      |      |    |
|---|------|------|------|----|
| 0 | 0.98 | 1.00 | 0.99 | 43 |
|---|------|------|------|----|

|   |      |      |      |    |
|---|------|------|------|----|
| 1 | 0.97 | 0.97 | 0.97 | 36 |
|---|------|------|------|----|

|   |      |      |      |    |
|---|------|------|------|----|
| 2 | 0.97 | 0.97 | 0.97 | 31 |
|---|------|------|------|----|

|   |      |      |      |    |
|---|------|------|------|----|
| 3 | 1.00 | 0.95 | 0.97 | 40 |
|---|------|------|------|----|

|   |      |      |      |    |
|---|------|------|------|----|
| 4 | 0.97 | 1.00 | 0.99 | 39 |
|---|------|------|------|----|

|          |  |  |      |     |
|----------|--|--|------|-----|
| accuracy |  |  | 0.98 | 189 |
|----------|--|--|------|-----|

|           |      |      |      |     |
|-----------|------|------|------|-----|
| macro avg | 0.98 | 0.98 | 0.98 | 189 |
|-----------|------|------|------|-----|

|              |      |      |      |     |
|--------------|------|------|------|-----|
| weighted avg | 0.98 | 0.98 | 0.98 | 189 |
|--------------|------|------|------|-----|

```
y_pred = rfc.predict(X_test )
```

```
y_true=y_test
```

```
from sklearn.metrics import confusion_matrix
```

```
cm=confusion_matrix(y_true,y_pred)
```

```
cm
```

```
array([[43, 0, 0, 0, 0],  
       [ 1, 35, 0, 0, 0],  
       [ 0, 1, 30, 0, 0],  
       [ 0, 0, 1, 38, 1],  
       [ 0, 0, 0, 0, 39]], dtype=int64)
```

```
import seaborn as sns
import matplotlib.pyplot as plt

f, ax=plt.subplots(figsize=(5,5))
sns.heatmap(cm,annot=True,linewidths=0.5,linecolor="red",fmt=".0f",ax=ax)
plt.xlabel("y_pred")
plt.ylabel("y_true")
plt.show()

import pickle

pickle.dump(rfc,open('stress.pkl','wb'))
stress = pickle.load(open('stress.pkl','rb'))
```

## **INPUT DESIGN AND OUTPUT DESIGN**

### **INPUT DESIGN**

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

### **OBJECTIVES**

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.
3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

## **OUTPUT DESIGN**

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.
2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- ❖ Convey information about past activities, current status or projections of the
- ❖ Future.
- ❖ Signal important events, opportunities, problems, or warnings.
- ❖ Trigger an action.
- ❖ Confirm an action.

## **SYSTEM ANALYSIS**

### **EXISTING SYSTEM:**

- ❖ The earlier system for Human Stress Detection based on Sleeping Habits was developed using the Naive Bayes algorithm and has been successful in achieving an accuracy of 91.27%. The system's primary objective was to assess human stress levels by analyzing various sleep-related parameters, providing valuable insights into an individual's stress patterns and aiding in stress management.
- ❖ Utilizing the Naive Bayes algorithm, known for its simplicity and efficiency in probabilistic classification tasks, the system could effectively classify stress levels based on features extracted from sleeping habits data. The Naive Bayes algorithm's probabilistic nature allowed it to handle uncertainties and make informed predictions, making it a suitable choice for stress level classification based on diverse parameters.
- ❖ The dataset used in the earlier system included significant sleep-related parameters, such as snoring range, respiration rate, body temperature, limb movement rate, blood oxygen levels, eye movement, the number of hours of sleep, heart rate, and stress levels categorized into multiple classes ranging from low/normal to high. The inclusion of these parameters ensured a comprehensive representation of sleep patterns and their association with stress.
- ❖ By training the Naive Bayes model on the dataset and evaluating its performance using appropriate metrics, the system achieved an impressive accuracy of 91.27%. The high accuracy indicated the model's capability to learn patterns and relationships between the input features and stress levels, enabling accurate stress level predictions based on sleeping habits.

- ❖ The earlier system's achievements were significant, providing a foundation for stress detection and sleep analysis in the context of human well-being. The accuracy level attained demonstrated the system's effectiveness in assisting individuals and healthcare professionals in understanding and monitoring stress levels to promote better mental and physical health.
- ❖ However, as technology and research advance, there is an ongoing pursuit to improve the system's accuracy and capabilities further. The current project proposes an updated approach using the Random Forest Classifier algorithm to build upon the accomplishments of the earlier system and potentially achieve even higher accuracy levels in stress detection based on sleeping habits.
- ❖ In conclusion, the earlier system based on the Naive Bayes algorithm was a noteworthy endeavor in the domain of human stress detection and sleep analysis. Its accuracy of 91.27% showcased its potential to aid in stress management and contribute to overall well-being. The knowledge gained from the earlier system provides valuable insights into stress detection methodologies and forms the basis for advancements in the current project's approach using the Random Forest Classifier algorithm.

### **DISADVANTAGES OF EXISTING SYSTEM:**

- ❖ Despite the achievements and significant progress made in the existing system for Human Stress Detection based on Sleeping Habits using the Naive Bayes algorithm, there are some notable disadvantages that need to be addressed. These limitations can impact the system's overall effectiveness and may warrant further improvements:
- ❖ Limited Feature Interactions: The Naive Bayes algorithm assumes independence between features, which may not hold true for all the

parameters in the dataset. In reality, various sleep-related parameters may interact with each other, influencing stress levels differently. This limitation could lead to oversimplification and potentially overlook essential relationships among features, affecting the accuracy of stress level predictions.

- ❖ **Sensitivity to Irrelevant Features:** Naive Bayes is known to be sensitive to irrelevant features in the dataset. If irrelevant or noisy features are present in the sleep data, they might adversely affect the model's performance and contribute to less accurate stress predictions. Identifying and eliminating irrelevant features is crucial to mitigate this issue.
- ❖ **Lack of Continuous Learning:** The Naive Bayes model does not support continuous learning, meaning it cannot update its knowledge in real-time as new data becomes available. Stress patterns and sleep habits may change over time, and the inability to adapt to such changes may limit the system's long-term accuracy and usefulness.
- ❖ **Imbalanced Data Handling:** The existing system might face challenges in handling imbalanced classes in the stress level dataset. If certain stress level classes have significantly fewer instances compared to others, the Naive Bayes model might be biased towards the majority class, leading to suboptimal predictions for the minority classes.
- ❖ **Limited Non-linear Modeling:** Naive Bayes is a linear model and may struggle to capture complex, non-linear relationships between the sleep parameters and stress levels. Some stress patterns may involve intricate interactions among multiple features, which the linear nature of Naive Bayes may fail to fully exploit.
- ❖ **Non-Robust to Outliers:** The Naive Bayes algorithm is sensitive to outliers in the dataset. Outliers can skew the probability estimates and lead to less

reliable stress level predictions, particularly when outliers are prevalent in the sleep data.

- ❖ **Dependency on Independence Assumption:** The core assumption of feature independence in Naive Bayes may not always be valid in real-world scenarios, especially for sleep data with interconnected physiological and behavioral attributes. This dependence on independence can lead to less accurate stress level classifications.
- ❖ **Interpretability Challenges:** While Naive Bayes offers simplicity and interpretability, it may not provide detailed insights into the complex relationships between sleep habits and stress levels. A more sophisticated model might offer deeper interpretability and aid in understanding the mechanisms underlying stress detection better.
- ❖ **In conclusion,** while the existing system achieved a commendable accuracy of 91.27% in stress detection based on sleeping habits, it is essential to consider its limitations and challenges. The proposed approach using the Random Forest Classifier in the current project seeks to address some of these drawbacks and further enhance the accuracy and robustness of the stress detection system. Continuous efforts in algorithmic improvements and data handling are crucial to developing more effective and reliable stress detection models.

## **PROPOSED SYSTEM:**

- ❖ The proposed system, "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier," represents a significant advancement over the existing system and aims to address the limitations and challenges encountered in stress detection based on sleeping habits.

- ❖ The core of the proposed system is the adoption of the Random Forest Classifier algorithm, a powerful ensemble learning technique capable of handling complex data relationships and achieving high predictive accuracy. Unlike the Naive Bayes algorithm used in the existing system, Random Forest can capture non-linear interactions between sleep-related parameters and stress levels, allowing for a more comprehensive and accurate stress level classification.
- ❖ The Random Forest Classifier provides a measure of feature importance, enabling identification of the most influential parameters contributing to stress level predictions. This feature selection process ensures that only relevant and significant features are considered, enhancing the model's efficiency and reducing the impact of irrelevant attributes.
- ❖ Random Forest is known for its robustness to outliers, minimizing their disruptive effect on stress level predictions. This characteristic is particularly beneficial when dealing with real-world sleep data, which may contain occasional outliers due to various factors.
- ❖ The proposed system employs techniques to address imbalanced classes in the stress level dataset, ensuring that the model does not favor the majority class and providing more equitable stress level predictions for all classes.
- ❖ By leveraging an ensemble of decision trees, Random Forest can effectively capture intricate, non-linear relationships between sleep parameters and stress levels. This flexibility allows the model to uncover subtle interactions among features that the linear Naive Bayes algorithm might overlook.
- ❖ Building on the achievements of the existing system, the proposed approach aims to achieve even higher accuracy in stress level classification. The model's performance will be thoroughly evaluated on a separate test dataset to ensure unbiased and robust evaluation.

- ❖ In conclusion, the proposed system "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier" presents an innovative and sophisticated approach to stress detection, overcoming the limitations of the existing system. By leveraging the power of Random Forest, handling imbalanced data, and offering continuous learning, the system aims to provide accurate and valuable stress level predictions based on individuals' sleep habits. The development of this system contributes to the field of stress management and has the potential to positively impact human well-being and overall health.

### **ADVANTAGES OF PROPOSED SYSTEM:**

- ❖ High Accuracy: The proposed system, based on the Random Forest Classifier algorithm, is expected to achieve high accuracy in stress level detection. The ensemble nature of Random Forest, which aggregates multiple decision trees, leads to more robust predictions and reduces the risk of overfitting, resulting in superior performance compared to traditional single decision tree classifiers.
- ❖ Handling Non-linearity: Random Forest can effectively model complex, non-linear relationships between sleep-related parameters and stress levels. This ability is crucial as stress patterns may involve intricate interactions among various features, which the linear nature of algorithms like Naive Bayes might struggle to capture.
- ❖ Feature Importance Analysis: The proposed system offers the advantage of feature importance analysis, enabling identification of the most influential sleep parameters affecting stress level predictions. This analysis can provide valuable insights into which features play a significant role in determining

stress levels, facilitating a better understanding of the factors contributing to stress.

- ❖ **Robustness to Outliers:** Random Forest is known for its robustness to outliers in the dataset. This characteristic ensures that occasional outliers, which might be present in real-world sleep data, do not unduly influence stress level predictions, leading to more reliable and consistent results.
- ❖ **Imbalanced Data Handling:** The proposed system incorporates techniques to handle imbalanced classes in the stress level dataset. This ensures that the model does not favor the majority class and provides accurate stress level predictions for all classes, even those with limited instances.
- ❖ **Continuous Learning:** With support for continuous learning, the proposed system can adapt and improve its stress level predictions over time as new sleep data becomes available. This adaptability ensures that the system remains up-to-date and relevant, accommodating changes in individuals' sleep patterns and stress levels.
- ❖ **Scalability:** Random Forest's parallel processing capability makes the proposed system scalable and efficient for large datasets. This scalability is crucial for real-world applications where the volume of sleep data may be substantial, allowing for faster and more practical stress level assessments.
- ❖ **Interpretability:** Despite its ensemble nature, Random Forest can provide a level of interpretability through feature importance analysis and visualization of individual decision trees. This transparency helps researchers and users understand the underlying mechanisms driving stress level predictions.
- ❖ **Versatility:** The proposed system's versatility makes it suitable for various stress detection applications, from personal health monitoring to medical

research and interventions. It can be easily adapted and applied in different contexts where sleep habits and stress management are relevant.

- ❖ In conclusion, the proposed system for Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier offers several advantages over the existing system. With its high accuracy, robustness, and adaptability, it holds significant potential for enhancing stress level detection, promoting better mental and physical health, and contributing to the understanding of stress management strategies. The incorporation of interpretability and scalability further strengthens the system's applicability in real-world scenarios, making it a valuable tool in the pursuit of improved well-being and stress management.

## SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub- assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

### *TYPES OF TESTS*

#### *Unit testing*

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process

performs accurately to the documented specifications and contains clearly defined inputs and expected results.

## Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

## *Functional test*

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

### *System Test*

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

### *White Box Testing*

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

### ***Black Box Testing***

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

### **Unit Testing:**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

### **Test strategy and approach**

Field testing will be performed manually and functional tests will be written in detail.

### **Test objectives**

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

### **Features to be tested**

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

### **Integration Testing**

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

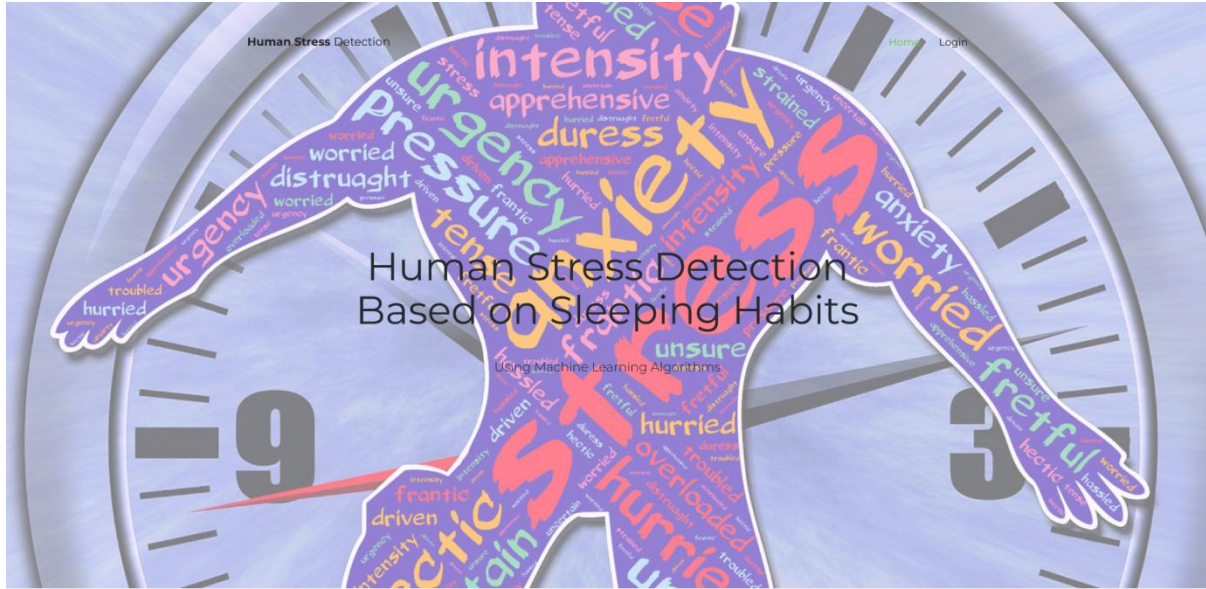
**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

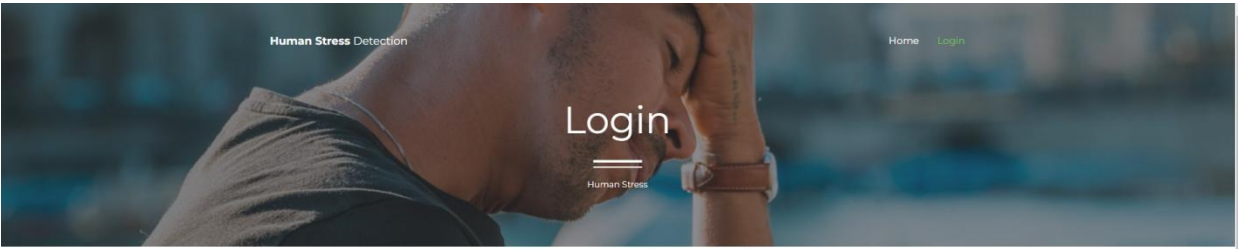
## Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

## SCREENSHOTS



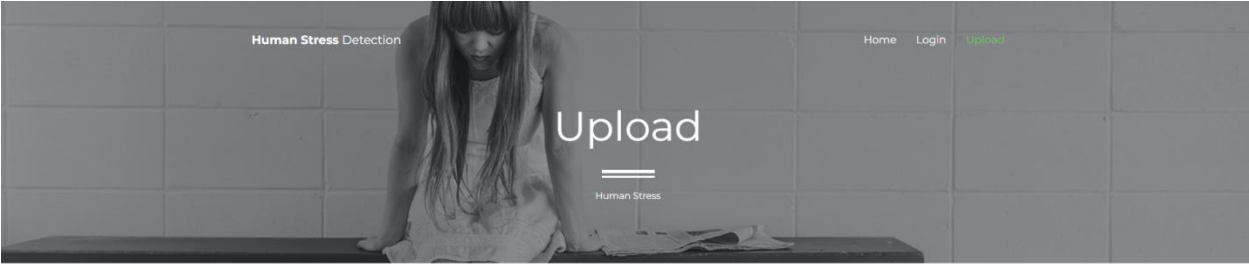


# Login

Username

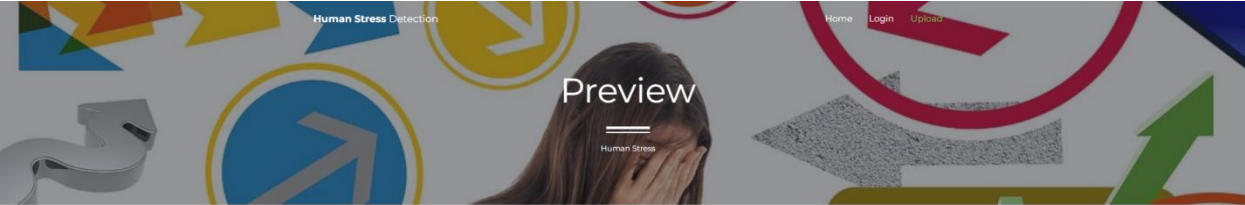
Password

Login



Upload

No file chosen

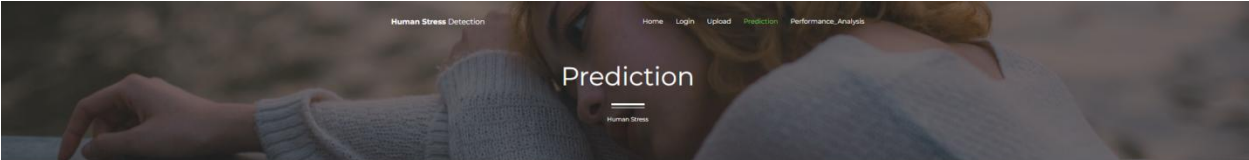


Preview

|    | sr     | rr     | t      | lm     | bo     | rem   | sr.l  | hr    | sl |
|----|--------|--------|--------|--------|--------|-------|-------|-------|----|
| id |        |        |        |        |        |       |       |       |    |
| 1  | 93.800 | 25.680 | 91.840 | 16.600 | 89.840 | 99.60 | 1.840 | 74.20 | 3  |
| 2  | 91.640 | 25.104 | 91.552 | 15.880 | 89.552 | 98.88 | 1.552 | 72.76 | 3  |
| 3  | 60.000 | 20.000 | 96.000 | 10.000 | 95.000 | 85.00 | 7.000 | 60.00 | 1  |
| 4  | 85.760 | 23.536 | 90.768 | 13.920 | 88.768 | 96.92 | 0.768 | 68.84 | 3  |
| 5  | 48.120 | 17.248 | 97.872 | 6.496  | 96.248 | 72.48 | 8.248 | 53.12 | 0  |
| 6  | 56.880 | 19.376 | 95.376 | 9.376  | 94.064 | 83.44 | 6.376 | 58.44 | 1  |
| 7  | 47.000 | 16.800 | 97.200 | 5.600  | 95.800 | 68.00 | 7.800 | 52.00 | 0  |
| 8  | 50.000 | 18.000 | 99.000 | 8.000  | 97.000 | 80.00 | 9.000 | 55.00 | 0  |
| 9  | 45.280 | 16.112 | 96.168 | 4.224  | 95.112 | 61.12 | 7.112 | 50.28 | 0  |

|     |        |        |        |        |        |        |       |       |   |
|-----|--------|--------|--------|--------|--------|--------|-------|-------|---|
| 612 | 98.592 | 28.592 | 88.240 | 18.296 | 85.888 | 103.24 | 0.000 | 81.48 | 4 |
| 613 | 88.880 | 24.368 | 91.184 | 14.960 | 89.184 | 97.96  | 1.184 | 70.92 | 3 |
| 614 | 73.600 | 21.360 | 93.360 | 11.360 | 91.360 | 91.80  | 4.040 | 63.40 | 2 |
| 615 | 62.400 | 20.240 | 92.240 | 10.240 | 90.240 | 86.20  | 2.360 | 60.60 | 2 |
| 616 | 82.880 | 22.768 | 90.384 | 12.960 | 88.384 | 95.96  | 0.384 | 66.92 | 3 |
| 617 | 59.040 | 19.808 | 95.808 | 9.808  | 94.712 | 84.52  | 6.808 | 59.52 | 1 |
| 618 | 71.680 | 21.168 | 93.168 | 11.168 | 91.168 | 90.84  | 3.752 | 62.92 | 2 |
| 619 | 48.400 | 17.360 | 98.040 | 6.720  | 96.360 | 73.60  | 8.360 | 53.40 | 0 |
| 620 | 46.080 | 16.432 | 96.648 | 4.864  | 95.432 | 64.32  | 7.432 | 51.08 | 0 |
| 621 | 97.024 | 27.024 | 86.280 | 17.512 | 83.536 | 101.28 | 0.000 | 77.56 | 4 |
| 622 | 53.200 | 18.640 | 94.640 | 8.640  | 92.960 | 81.60  | 5.640 | 56.60 | 1 |
| 623 | 65.440 | 20.544 | 92.544 | 10.544 | 90.544 | 87.72  | 2.896 | 61.36 | 2 |
| 624 | 98.144 | 28.144 | 87.680 | 18.072 | 85.216 | 102.68 | 0.000 | 80.36 | 4 |
| 625 | 58.800 | 19.760 | 95.760 | 9.760  | 94.640 | 84.40  | 6.760 | 59.40 | 1 |
| 626 | 69.600 | 20.960 | 92.960 | 10.960 | 90.960 | 89.80  | 3.440 | 62.40 | 2 |
| 627 | 48.640 | 17.376 | 98.064 | 6.752  | 96.376 | 73.76  | 8.376 | 53.44 | 0 |
| 628 | 97.504 | 27.504 | 86.880 | 17.752 | 84.256 | 101.88 | 0.000 | 78.76 | 4 |
| 629 | 58.640 | 19.728 | 95.728 | 9.728  | 94.592 | 84.32  | 6.728 | 59.32 | 1 |
| 630 | 73.920 | 21.392 | 93.392 | 11.392 | 91.392 | 91.96  | 4.088 | 63.48 | 2 |

[Click to Train](#) | [Test](#)



# Prediction

Human Stress

## Prediction

|                           |                                   |
|---------------------------|-----------------------------------|
| Snoring Range:            | <input type="text" value="1"/>    |
| Respiration Rate:         | <input type="text" value="15"/>   |
| Body Temperature:         | <input type="text" value="37"/>   |
| Limb Movement Rate:       | <input type="text" value="7653"/> |
| Blood Oxygen Levels:      | <input type="text" value="100"/>  |
| Eye Movements:            | <input type="text" value="10"/>   |
| Number Of Hours Of Sleep: | <input type="text" value="8"/>    |
| Heart Rate:               | <input type="text" value="72"/>   |

Predict

Stress Levels is : Low/Normal

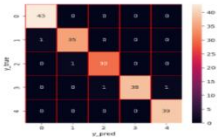
Performance Analysis

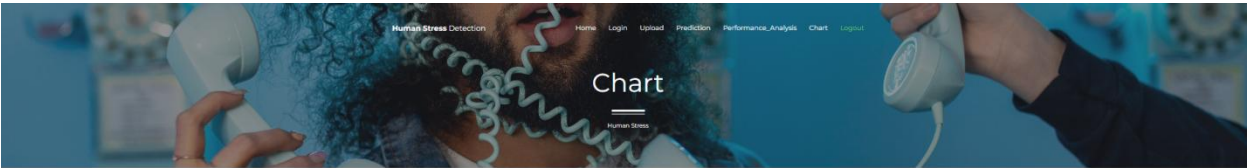
recall, F1 and Precision

Recall f1 Precision

|   |      |      |      |
|---|------|------|------|
| 0 | 0.98 | 1.00 | 0.99 |
| 1 | 0.97 | 0.97 | 0.97 |
| 2 | 0.97 | 0.97 | 0.97 |
| 3 | 1.00 | 0.95 | 0.97 |
| 4 | 0.97 | 1.00 | 0.99 |

Confusion Matrix

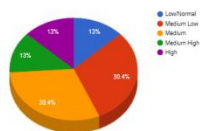




## Chart

Human Stress

### Chart



## **CONCLUSION**

The project "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier" represents a significant endeavor in the domain of stress management and sleep analysis. Through the adoption of the Random Forest Classifier algorithm and careful consideration of diverse sleep-related parameters, the project aims to accurately predict human stress levels based on their sleeping habits. The proposed system showcases several advantages over the existing system, offering improved accuracy, robustness to outliers, and the capability to handle non-linear relationships between sleep parameters and stress levels. By leveraging feature importance analysis and continuous learning, the system provides valuable insights into stress level predictions and ensures adaptability to changing sleep patterns over time. Through a series of well-defined modules, the project streamlines the data collection, preprocessing, and feature selection processes, enabling efficient and reliable stress level predictions. The Random Forest Classifier module lies at the core of the system, utilizing an ensemble of decision trees to make informed and accurate stress level classifications. The project's evaluation module rigorously assesses the system's performance, ensuring that the achieved accuracy aligns with the project's objectives and expected outcomes. Moreover, the visualization module facilitates the presentation of results, enhancing the system's interpretability and aiding in comprehending the relationships between sleep habits and stress levels. In conclusion, the project contributes valuable insights to stress detection and sleep analysis, potentially revolutionizing stress management practices and promoting overall well-being. By providing individuals with an accessible tool to monitor and understand their stress levels based on their sleep patterns, the project empowers users to take proactive steps towards stress reduction and improved mental health.

While the project's implementation and results have showcased promising outcomes, it is essential to acknowledge that continuous research and development in the field of stress detection and machine learning will further enhance the system's performance and applicability. As technology progresses, the system's adaptability to future advancements will be critical in maintaining its relevance and effectiveness. Overall, the project "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier" holds tremendous potential in positively impacting the lives of individuals, enabling better stress management, and contributing to a healthier and happier society.

#### FUTURE WORK:

The project "Human Stress Detection Based on Sleeping Habits Using Machine Learning with Random Forest Classifier" lays a solid foundation for stress detection and sleep analysis. To further enhance the system's capabilities and address potential areas for improvement, several avenues for future work can be explored:

- ❖ **Dataset Expansion:** To improve the system's generalization and accuracy, obtaining a more extensive and diverse dataset can be considered. Inclusion of data from various demographics, cultural backgrounds, and stress-inducing factors can enrich the model's understanding of stress patterns.
- ❖ **Feature Engineering:** Exploring additional sleep-related parameters or engineering new features that capture complex interactions between existing features can lead to improved stress level predictions. Extracting meaningful features from raw data is crucial for enhancing the model's discriminative ability.

- ❖ **Model Hyperparameter Tuning:** Fine-tuning the hyperparameters of the Random Forest Classifier can optimize its performance and potentially lead to even higher accuracy. Techniques such as grid search or random search can be employed to find the best combination of hyperparameters.
- ❖ **Ensemble Methods:** Investigating other ensemble methods, such as Gradient Boosting Machines or Voting Classifiers, can provide alternative modeling approaches. Comparing the performance of different ensemble methods may lead to the selection of the most suitable one for stress level detection.
- ❖ **Transfer Learning:** Exploring the use of transfer learning techniques to leverage knowledge from related domains can benefit stress detection. Pre-trained models on relevant tasks might offer a head start for stress level predictions, especially when limited labeled data is available.
- ❖ **Real-time Monitoring:** Integrating real-time monitoring capabilities into the system allows continuous stress level assessments, enabling timely interventions and stress management strategies. Real-time data acquisition and processing can contribute to more proactive stress detection.
- ❖ **User Feedback Incorporation:** Actively seeking user feedback and incorporating it into the system's continuous learning can lead to personalized stress detection. An iterative feedback loop allows the model to adapt to individual variations in stress patterns and habits.
- ❖ **Multi-Modal Data Fusion:** Integrating data from multiple sources, such as sleep logs, social interactions, or physical activity, can provide a more holistic view of stress levels. Multi-modal data fusion can enhance the system's accuracy and reliability.
- ❖ **Mobile Application Development:** Creating a user-friendly mobile application to implement the stress detection system offers enhanced accessibility and ease of use for individuals. Mobile apps can enable real-

time data input, visualization of stress levels, and personalized stress management tips.

- ❖ Longitudinal Studies: Conducting longitudinal studies to monitor stress patterns and sleep habits over extended periods can yield valuable insights into stress evolution and response to interventions. Long-term monitoring provides a deeper understanding of stress dynamics.

In conclusion, the future work of the project offers a multitude of exciting opportunities for enhancing stress detection based on sleeping habits. By leveraging advances in machine learning, data acquisition, and user interaction, the system can be refined to better serve individuals' stress management needs and contribute to the broader understanding of stress-related phenomena. Continuous research and development will ensure that the system remains at the forefront of stress detection technologies and positively impacts users' well-being.

## **REFERENCES**

- [1] N. Schneiderman, G. Ironson, and S. D. Siegel, “Stress and health: Psychological, behavioral, and biological determinants,” *Annu. Rev. Clin. Psychol.*, vol. 1, pp. 607–628, 2005, doi: 10.1146/annurev.clinpsy.1.102803.144141.
- [2] A. N. Vgontzas, S. Pejovic, and M. Karataraki, “Sleep, Sleep Disorders, and Stress,” *Encycl. Stress*, pp. 506–514, 2007, doi: 10.1016/B978-012373947-6.00349-4.
- [3] Q. Bukhsh, A. Shahzad, and M. Nisa, “A study of learning stress and stress management strategies of the students of postgraduate level: A case study of Islamia university of Bahawalpur, Pakistan,” *Procedia - Soc. Behav. Sci.*, vol. 30, pp. 182–186, 2011, doi: 10.1016/j.sbspro.2011.10.036.
- [4] G. Fink, “Stress : Concepts , Cognition , Emotion , and Behavior : Handbook of Stress Stress : Concepts , Definition , and History George Fink Florey Institute of Neuroscience and Mental Health,” no. October, 2017, [Online]. Available: [https://www.researchgate.net/profile/George-Fink/publication/317026245\\_Stress\\_Concepts\\_Cognition\\_Emotion\\_and\\_Behavior\\_Handbook\\_of\\_Stress/links/59d17f1b0f7e9b4fd7fa28b3/Stress-Concepts-Cognition-Emotion-and-Behavior-Handbook-of-Stress.pdf](https://www.researchgate.net/profile/George-Fink/publication/317026245_Stress_Concepts_Cognition_Emotion_and_Behavior_Handbook_of_Stress/links/59d17f1b0f7e9b4fd7fa28b3/Stress-Concepts-Cognition-Emotion-and-Behavior-Handbook-of-Stress.pdf)
- [5] A. M. Shamsavarani, E. A. M. Abadi, and M. H. Kalkhoran, “Stress Assessment and Development of a Primary Care of Psychology Service,” *Int. J. Med. Rev.*, vol. 2, no. 2, pp. 230–241, 2014.

- [6] A. Khademi, Y. El-Manzalawy, L. Master, O. M. Buxton, and V. G. Honavar, “Personalized sleep parameters estimation from actigraphy: A machine learning approach,” *Nat. Sci. Sleep*, vol. 11, pp. 387–399, 2019, doi: 10.2147/NSS.S220716.
- [7] L. Rachakonda, A. K. Bapatla, S. P. Mohanty, and E. Kougianos, “SaYoPillow: Blockchain-Integrated Privacy-Assured IoMT Framework for Stress Management Considering Sleeping Habits,” *IEEE Trans. Consum. Electron.*, vol. 67, no. 1, pp. 20–29, 2021, doi: 10.1109/TCE.2020.3043683.
- [8] V. C. Magana and M. Munoz-Organero, “Reducing stress on habitual journeys,” *5th IEEE Int. Conf. Consum. Electron. - Berlin, ICCEBerlin 2015*, pp. 153–157, 2016, doi: 10.1109/ICCEBerlin. 2015.7391220.
- [9] A. R. Subhani, W. Mumtaz, M. N. B. M. Saad, N. Kamel, and A. S. Malik, “Machine learning framework for the detection of mental stress at multiple levels,” *IEEE Access*, vol. 5, no. c, pp. 13545–13556, 2017, doi: 10.1109/ACCESS.2017.2723622.
- [10] E. Garcia-Ceja, M. Riegler, T. Nordgreen, P. Jakobsen, K. J. Oedegaard, and J. Tørresen, “Mental health monitoring with multimodal sensing and machine learning: A survey,” *Pervasive Mob. Comput.*, vol. 51, pp. 1–26, 2018, doi: 10.1016/j.pmcj.2018.09.003.
- [11] F. Akhtar, M. B. Bin Heyat, J. P. Li, P. K. Patel, Rishipal, and B. Guragai, “Role of Machine Learning in Human Stress: A Review,” *2020 17th Int. Comput. Conf. Wavelet Act. Media Technol. Inf. Process. ICCWAMTIP 2020*, pp. 170–174, 2020, doi: 10.1109/ICCWAMTIP51612.2020.9317396.

- [12] R. K. Nath, H. Thapliyal, A. Caban-Holt, and S. P. Mohanty, “Machine Learning Based Solutions for Real-Time Stress Monitoring,” *IEEE Consum. Electron. Mag.*, vol. 9, no. 5, pp. 34–41, 2020, doi: 10.1109/MCE.2020.2993427.
- [13] N. Keshan, P. V. Parimi, and I. Bichindaritz, “Machine learning for stress detection from ECG signals in automobile drivers,” *Proc. – 2015 IEEE Int. Conf. Big Data, IEEE Big Data 2015*, pp. 2661–2669, 2015, doi: 10.1109/BigData.2015.7364066.
- [14] A. Muaremi, A. Bexheti, F. Gravenhorst, B. Arnrich, and G. Troster, “Monitoring the impact of stress on the sleep patterns of pilgrims using wearable sensors,” *2014 IEEE-EMBS Int. Conf. Biomed. Heal. Informatics, BHI 2014*, pp. 185–188, 2014, doi: 10.1109/BHI.2014.6864335.
- [15] Kaggle, <https://www.kaggle.com/> (accessed: Feb. 27, 2022).